

The Physics of LLM Inference

Elliot Arledge

Contents

Introduction	3
What We Build	3
How to Read This Book	4
Chapter 1: Transformer Mechanics	6
Attention from Scratch	6
Multi-Head and GQA	7
The Feedforward Network	9
Putting It Together	10
Chapter 2: The Generation Loop	13
Autoregressive Generation	13
The Prefill Phase	14
The Decode Phase	15
The KV Cache	17
Memory Math	18
Chapter 3: Compute Characteristics	21
GEMM	21
GEMV	22
The Roofline Model	24
Batching Effects	25
Chapter 4: Kernel Fundamentals	28
GPU Architecture	28
Hello CUDA	30
Python Bindings	31
Profiling with nsys	33
Launch Overhead	34
Chapter 5: Kernel Optimization	36
Memory Coalescing	36
Shared Memory	37
Tensor Cores	39
Profiling with ncU	40
Triton	41
PTX and SASS	43

Chapter 6: FlashAttention	46
The Attention Bottleneck	46
Online Softmax	47
Tiled Attention	49
FlashInfer	50
Chapter 7: Continuous Batching	52
Static Batching	52
The Orca Insight	53
Scheduler Design	55
Radix Cache	56
Memory Management	58
Chapter 8: Advanced Scheduling	60
Chunked Prefill	60
Mixed Batches	61
CUDA Graphs	63
Overlap Scheduling	65
Chapter 9: Mixture of Experts	68
MoE Basics	68
MoE Inference	70
Chapter 10: Production Server	73
API Design	73
FastAPI Server	76
Tokenizer Workers	78
Putting It Together	81
Benchmarking	85
Appendix	88
What’s Next	88
Speculative Decoding Preview	89
Further Reading	91

Introduction

You download a 7B model from Hugging Face. It's 14 GB in fp16. Your GPU has 8 GB of VRAM. You run the script and get **CUDA out of memory**. You try quantization, offloading, every trick you can find on Reddit. Maybe you get it running at 2 tokens per second. This is the first wall.

You rent a bigger GPU. Now the model fits, runs at 50 tokens per second for a single user. You open it up to friends. Ten people hit it at once. Latency spikes to 30 seconds. The twenty-first request times out. This is the second wall.

The gap between “I downloaded a model” and “I’m running a production inference service” is enormous. It’s not about having more GPUs - it’s about understanding the physics of the system: where memory goes, where compute goes, why naive approaches collapse under load, and what the clever solutions actually do.

This book closes that gap on a single GPU. We’ll start with one GPU and one request, then systematically tackle each bottleneck: the KV cache that makes autoregressive generation feasible, the memory bandwidth wall that dominates decode performance, the scheduling algorithms that keep GPUs saturated under concurrent load, the kernel optimizations that squeeze every FLOP out of the hardware.

By the end, you’ll understand the physics of LLM inference - the fundamental constraints and techniques that apply whether you’re running a 7B model on a consumer GPU or a mixture-of-experts model on an H100. Not magic, but engineering constrained by physics.

What We Build

By the end of this book, you’ll have built a complete LLM inference server from scratch. Not a toy demo - a real system with continuous batching, paged memory, and optimized CUDA kernels. The kind of system that can saturate an H100 and serve hundreds of concurrent requests.

The reference implementation is mini-sglang, a stripped-down version of the SGLang serving framework. About 5,000 lines of Python. Small enough to understand completely, large enough to be useful.

Why build a serving system instead of just running inference? Because serving is where the interesting problems live. Single-user inference is straightforward: tokenize, forward pass, sample, detokenize. Add a thousand concurrent requests and everything changes. Memory becomes precious. Scheduling matters. The naive approach collapses under load.

The core components:

API layer: HTTP endpoints that accept prompts and stream tokens back. OpenAI-compatible, so existing client libraries work out of the box.

Tokenizer pool: CPU-bound tokenization runs in parallel worker processes. Keeps tokenization from blocking GPU work.

Scheduler: The brain of the system. Decides which requests run in each batch, manages memory allocation, handles preemption when resources get tight. This is where continuous batching lives.

Engine: Runs the actual forward passes. Manages the KV cache, executes CUDA graphs for decode steps, coordinates with the scheduler.

The target model throughout is Qwen3-30B-A3B, a mixture-of-experts architecture that demonstrates both dense and sparse inference patterns. But the techniques apply to any transformer: Qwen, Gemini, GPT, whatever comes next.

Each chapter explains one piece of this system until the whole thing clicks into place.

How to Read This Book

This book is meant to be read linearly, at least the first time through. Each chapter builds on the previous ones, and later optimizations only make sense once you understand what's being optimized.

The first two chapters are foundation. Chapter 1 reviews transformer mechanics with an implementation focus. If you already know attention and feedforward networks cold, skim it. If you're shaky on the details, take your time - everything that follows depends on understanding the building blocks.

Chapter 2 is the first major insight: the generation loop. Understanding prefill versus decode, and why the KV cache matters, is essential. This chapter is worth reading twice.

Chapters 3 through 6 are the kernel story. We start with compute characteristics - GEMM versus GEMV, the roofline model, why batch size matters. Then we dive into GPU architecture and CUDA basics. Chapter 5 is the second climax: kernel optimization through coalescing, shared memory, and tensor cores. Chapter 6 applies these ideas to FlashAttention.

Chapters 7 and 8 are the serving story. Chapter 7 introduces continuous batching, the insight from the Orca paper that transformed LLM serving. This is the third climax: understanding why iteration-level scheduling beats request-level scheduling. Chapter 8 covers advanced techniques like chunked prefill and CUDA graphs.

Chapter 9 covers mixture-of-experts, an architectural pattern that increases model capacity without proportional compute increase.

Chapter 10 brings everything together into a production server.

The code lives in a companion repository. Every chapter has working Python that you can run and modify. The code in the book is intentionally minimal - we show the core concept in as few lines as possible, then point to the repository for the complete implementation.

When you see a code block, type it out. Don't copy-paste. The muscle memory matters.

When you see a benchmark, run it on your own hardware. The numbers in this book come from an H100 SXM, but your mileage will vary. The relative comparisons matter more than the absolute numbers.

When something doesn't make sense, stop and figure it out. This book assumes you have PyTorch experience, but nothing else. CUDA knowledge, systems programming - we'll teach you what you need.

The three climax chapters are:

- Chapter 2: The KV cache insight
- Chapter 5: Kernel optimization (the magic of tiled matmul)
- Chapter 7: Continuous batching (why Orca changed everything)

These are the “aha” moments. If you get stuck elsewhere, push through to these chapters. Once you understand them, the rest falls into place.

Chapter 1: Transformer Mechanics

Before we can optimize inference, we need to understand what we're optimizing. This chapter builds the transformer from scratch, focusing on the operations that matter for inference performance.

If you've trained transformers before, this is review with a twist. We're not just implementing attention - we're implementing it in a way that makes the memory patterns visible. Every matrix multiply, every reshape, every memory allocation shows up later when we profile the system.

We'll build four components: the attention mechanism, multi-head attention with grouped-query attention, the feedforward network, and the full transformer block. Each piece is small enough to hold in your head, large enough to be the real thing.

The code in this chapter is pure PyTorch. No optimization tricks, no CUDA kernels, just the math translated directly into tensor operations. This is the baseline we'll spend the rest of the book beating.

Attention from Scratch

Attention is a weighted sum. That's the whole thing. Given a query, we compute how much attention to pay to each value, then sum the values weighted by those attention scores.

In this section we cover:

- The core attention formula and what each term means
- Scaled dot-product attention in code
- The causal mask and why it matters for generation

The formula is:

`Attention(Q, K, V) = softmax(QKT / sqrt(dk)) V`

Q is the query matrix: what we're looking for. K is the key matrix: what each position advertises about itself. V is the value matrix: what each position actually contains. The dot product QK^T measures how well each query matches each key. The softmax normalizes these scores to sum to 1. The final multiply with V produces a weighted combination of values.

The $\text{sqrt}(d_k)$ scaling prevents dot products from getting too large. When d_k is 128 and the entries of Q and K are roughly unit variance, the dot products have variance around 128. Large values push softmax into saturation where gradients vanish. Dividing by $\text{sqrt}(128)$ brings variance back to 1.

Here's the basic implementation:

```
def attention(q, k, v):
    d_k = q.shape[-1]
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)
    weights = torch.softmax(scores, dim=-1)
    return torch.matmul(weights, v)
```

Three lines. The matmul computes all query-key dot products at once. The softmax normalizes each row. The final matmul produces the output.

For autoregressive generation, we need a causal mask. Position i should only attend to positions 0 through i , not future positions. Without the mask, the model could cheat by looking at the answer.

```
def causal_attention(q, k, v):
    d_k = q.shape[-1]
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)
    mask = torch.triu(torch.ones(scores.shape[-1], scores.shape[-1]), diagonal=1).bool()
    scores = scores.masked_fill(mask, float('-inf'))
    return torch.matmul(torch.softmax(scores, dim=-1), v)
```

The mask is an upper triangular matrix of True values. We set those positions to negative infinity before softmax. After softmax, $\exp(-\infty) = 0$, so future positions contribute nothing.

Let's trace through the shapes for a concrete example. Say we have batch size 2, sequence length 512, and head dimension 64:

- Q: (2, 512, 64)
- K: (2, 512, 64)
- V: (2, 512, 64)
- scores = QK^T : (2, 512, 64) @ (2, 64, 512) = (2, 512, 512)
- weights: (2, 512, 512)
- output = weights @ V: (2, 512, 512) @ (2, 512, 64) = (2, 512, 64)

Notice that the attention matrix is 512 x 512. For each of the 512 positions, we compute attention weights over all 512 positions. This is the $O(n^2)$ memory cost that FlashAttention addresses later.

Multi-Head and GQA

Single-head attention has a problem: one set of query-key-value projections can only capture one type of relationship. Multi-head attention runs multiple attention operations in parallel, each with its own learned projections.

In this section we cover:

- Multi-head attention and why we split the embedding dimension
- The memory cost of multiple KV heads
- Grouped-query attention as a compromise between performance and memory

The standard approach splits the embedding dimension across heads. If we have `hidden_dim=4096` and `num_heads=32`, each head works with `head_dim=128`. We project the input into Q, K, V for all heads at once, reshape to separate the heads, run attention independently, then concatenate and project back.

```
class MultiHeadAttention(nn.Module):
    def __init__(self, hidden_dim, num_heads):
        self.num_heads, self.head_dim = num_heads, hidden_dim // num_heads
        self.q_proj = nn.Linear(hidden_dim, hidden_dim)
        self.k_proj = nn.Linear(hidden_dim, hidden_dim)
        self.v_proj = nn.Linear(hidden_dim, hidden_dim)
        self.out_proj = nn.Linear(hidden_dim, hidden_dim)
```

The forward pass reshapes Q, K, V from (batch, seq, hidden) to (batch, heads, seq, head_dim), runs attention, then reshapes back:

```
def forward(self, x):
    B, S, _ = x.shape
    q = self.q_proj(x).view(B, S, self.num_heads, self.head_dim).transpose(1, 2)
    k = self.k_proj(x).view(B, S, self.num_heads, self.head_dim).transpose(1, 2)
    v = self.v_proj(x).view(B, S, self.num_heads, self.head_dim).transpose(1, 2)
    out = causal_attention(q, k, v)
    return self.out_proj(out.transpose(1, 2).contiguous().view(B, S, -1))
```

The transpose moves the head dimension before the sequence dimension: (batch, seq, heads, head_dim) becomes (batch, heads, seq, head_dim). This layout lets us batch the attention computation across heads.

The problem is memory. During inference, we cache K and V tensors to avoid recomputing them. With 32 heads and 128 dimensions per head, that's 4096 floats per token per layer. For a 70B model with 80 layers and 32K context, the KV cache alone is:

$2 \text{ (K + V)} * 80 \text{ layers} * 32 \text{ heads} * 128 \text{ dim} * 32768 \text{ tokens} * 2 \text{ bytes} = 42 \text{ GB}$

This is where grouped-query attention (GQA) helps. Instead of giving each query head its own KV head, we share KV heads across groups of query heads. Qwen3-72B uses 8 KV heads shared across 64 query heads. Each KV head serves 8 query heads.

```
class GroupedQueryAttention(nn.Module):
    def __init__(self, hidden_dim, num_q_heads, num_kv_heads):
        self.num_q_heads, self.num_kv_heads = num_q_heads, num_kv_heads
        self.num_groups = num_q_heads // num_kv_heads
        self.head_dim = hidden_dim // num_q_heads
        self.q_proj = nn.Linear(hidden_dim, num_q_heads * self.head_dim)
        self.k_proj = nn.Linear(hidden_dim, num_kv_heads * self.head_dim)
        self.v_proj = nn.Linear(hidden_dim, num_kv_heads * self.head_dim)
```

The key difference is fewer KV projections and `repeat_interleave` to expand K, V to match Q:

```
def forward(self, x):
    B, S, _ = x.shape
    q = self.q_proj(x).view(B, S, self.num_q_heads, self.head_dim).transpose(1, 2)
```

```

k = self.k_proj(x).view(B, S, self.num_kv_heads, self.head_dim).transpose(1, 2)
v = self.v_proj(x).view(B, S, self.num_kv_heads, self.head_dim).transpose(1, 2)
k = k.repeat_interleave(self.num_groups, dim=1)
v = v.repeat_interleave(self.num_groups, dim=1)

```

The key change is `repeat_interleave`. We project K and V to fewer heads, then expand them to match the query head count. The memory savings come from caching: we only store the smaller K and V tensors.

With 8 KV heads instead of 64, the cache shrinks by 8x:

$2 * 80 * 8 * 128 * 32768 * 2 \text{ bytes} = 5.2 \text{ GB}$

That’s the difference between fitting on one GPU or needing two.

The extreme case is multi-query attention (MQA) with a single KV head shared across all query heads. GQA is the middle ground: more expressive than MQA, more memory-efficient than full MHA.

The Feedforward Network

Attention moves information between positions. The feedforward network processes each position independently. If attention is about “what should I pay attention to,” the FFN is about “what should I do with what I found.”

In this section we cover:

- The structure of the feedforward network
- SwiGLU and gated activations
- Where the parameters actually live

The classic FFN is two linear layers with a nonlinearity:

```

class FeedForward(nn.Module):
    def __init__(self, hidden_dim, intermediate_dim):
        super().__init__()
        self.w1 = nn.Linear(hidden_dim, intermediate_dim)
        self.w2 = nn.Linear(intermediate_dim, hidden_dim)

    def forward(self, x):
        return self.w2(torch.relu(self.w1(x)))

```

The intermediate dimension is typically 4x the hidden dimension. For a 4096-dim model, the FFN expands to 16384 dimensions then contracts back. This expansion-contraction pattern is where the model stores most of its knowledge.

Modern transformers use gated activations instead of plain ReLU. Qwen3 uses SwiGLU, which splits the intermediate layer into two paths: one through a gate, one through the activation.

```

class SwiGLU(nn.Module):
    def __init__(self, hidden_dim, intermediate_dim):

```

```

    super().__init__()
    self.w1 = nn.Linear(hidden_dim, intermediate_dim, bias=False)
    self.w3 = nn.Linear(hidden_dim, intermediate_dim, bias=False)
    self.w2 = nn.Linear(intermediate_dim, hidden_dim, bias=False)

    def forward(self, x):
        return self.w2(F.silu(self.w1(x)) * self.w3(x))

```

The SiLU activation (also called Swish) is $x * \text{sigmoid}(x)$. The gate $w3(x)$ learns to modulate the activation, giving the network more control over information flow. The cost is an extra projection: three matrices instead of two.

To keep parameter count constant when switching from ReLU to SwiGLU, the intermediate dimension shrinks. Qwen3 uses $(2/3) * 4 * \text{hidden_dim}$ for the intermediate size, which after rounding gives a slight reduction.

Let's count parameters for a single transformer layer. With $\text{hidden_dim}=4096$, $\text{num_heads}=32$, $\text{head_dim}=128$, and $\text{intermediate_dim}=11008$:

Attention:

- Q projection: $4096 * 4096 = 16.7\text{M}$
- K projection: $4096 * 4096 = 16.7\text{M}$ (or less with GQA)
- V projection: $4096 * 4096 = 16.7\text{M}$ (or less with GQA)
- Output projection: $4096 * 4096 = 16.7\text{M}$
- Total attention: $\sim 67\text{M}$ (full MHA) or $\sim 38\text{M}$ (with 8 KV heads)

FFN (SwiGLU):

- $w1$: $4096 * 11008 = 45\text{M}$
- $w3$: $4096 * 11008 = 45\text{M}$
- $w2$: $11008 * 4096 = 45\text{M}$
- Total FFN: 135M

The FFN has roughly 2x the parameters of attention. For a 32-layer model, that's 4.3B parameters in FFN alone. This is where the knowledge lives.

During inference, the FFN is simple to optimize. Each token passes through independently with no dependencies on other tokens. This is embarrassingly parallel. The challenge is memory bandwidth: we need to load all those weights from GPU memory for every forward pass.

Putting It Together

A transformer block combines attention and FFN with residual connections and layer normalization. Stack N blocks, add an embedding layer and a language model head, and you have a complete model.

In this section we cover:

- The transformer block structure

- Pre-norm versus post-norm architectures
- The full decoder-only model

The standard transformer block:

```
class TransformerBlock(nn.Module):
    def __init__(self, hidden_dim, num_heads, intermediate_dim):
        self.norm1 = nn.RMSNorm(hidden_dim)
        self.attn = MultiHeadAttention(hidden_dim, num_heads)
        self.norm2 = nn.RMSNorm(hidden_dim)
        self.ffn = SwiGLU(hidden_dim, intermediate_dim)

    def forward(self, x):
        return x + self.ffn(self.norm2(x + self.attn(self.norm1(x))))
```

This is the pre-norm variant: normalize before each sublayer, then add the residual. The original transformer used post-norm (add then normalize), but pre-norm trains more stably at scale.

RMSNorm is simpler than LayerNorm. LayerNorm subtracts the mean and divides by standard deviation. RMSNorm just divides by the root mean square:

```
def rms_norm(x, weight, eps=1e-6):
    rms = torch.sqrt(torch.mean(x ** 2, dim=-1, keepdim=True) + eps)
    return weight * x / rms
```

No mean subtraction, no learnable bias. Fewer operations, same effect in practice.

The full model stacks blocks and wraps them with embedding and output layers:

```
class Transformer(nn.Module):
    def __init__(self, vocab_size, hidden_dim, num_layers, num_heads, intermediate_dim):
        self.embed = nn.Embedding(vocab_size, hidden_dim)
        self.layers = nn.ModuleList([TransformerBlock(...) for _ in range(num_layers)])
        self.norm = nn.RMSNorm(hidden_dim)
        self.lm_head = nn.Linear(hidden_dim, vocab_size, bias=False)
```

The forward pass embeds tokens, passes through all layers, and projects to vocabulary logits:

```
def forward(self, input_ids):
    x = self.embed(input_ids)
    for layer in self.layers:
        x = layer(x)
    return self.lm_head(self.norm(x))
```

The embedding layer maps token IDs to vectors. Each transformer block refines the representation. The final norm and lm_head map back to vocabulary logits.

A common optimization: tie the embedding and lm_head weights. Both are (vocab_size, hidden_dim) matrices, and sharing them reduces parameter count without hurting quality.

Let's trace a forward pass for a 7B model processing a 2048-token sequence:

1. **Embedding:** input_ids (batch, 2048) -> embeddings (batch, 2048, 4096)

2. **32 transformer blocks**, each:

- RMSNorm: (batch, 2048, 4096) -> (batch, 2048, 4096)
- Attention: computes 2048x2048 attention matrix per head
- Residual add
- RMSNorm
- FFN: (batch, 2048, 4096) -> (batch, 2048, 11008) -> (batch, 2048, 4096)
- Residual add

3. **Final norm**: (batch, 2048, 4096) -> (batch, 2048, 4096)

4. **LM head**: (batch, 2048, 4096) -> (batch, 2048, 32000)

The output is logits for all 32,000 vocabulary tokens at all 2048 positions. During training, we compute loss against all positions. During inference, we only care about the last position's logits.

This is the structure we're serving. Every optimization in this book targets some part of this forward pass: fusing operations, caching intermediate results, batching across requests, parallelizing across GPUs.

Chapter 2: The Generation Loop

This is where inference gets interesting.

Training a transformer is embarrassingly parallel: you have a batch of sequences, you compute the loss at all positions simultaneously, you backpropagate. The whole thing is one big matrix multiply after another.

Inference is different. You generate one token at a time, feed it back in, generate the next. Each new token depends on everything that came before. The naive approach recomputes the entire sequence at each step. The smart approach caches intermediate results.

This chapter introduces the fundamental distinction of LLM inference: prefill versus decode. The prefill phase processes your prompt in parallel, building up the KV cache. The decode phase generates tokens one at a time, reading from the cache. Understanding this split is essential for everything that follows.

The KV cache is the first big insight of this book. It transforms generation from $O(n^2)$ per token to $O(n)$ per token. It's also the source of most memory management complexity in production systems. We'll derive it from first principles, implement it, and calculate exactly how much memory it consumes.

Autoregressive Generation

Language models generate text one token at a time. Each token is sampled from a probability distribution conditioned on all previous tokens. This is autoregressive generation: the output at step t becomes part of the input at step $t+1$.

In this section we cover:

- The generation loop in its simplest form
- Why we can only generate one token per forward pass
- The computational cost of naive generation

Here's the basic generation loop:

```
def generate(model, prompt_ids, max_tokens):
    input_ids = prompt_ids.clone()
    for _ in range(max_tokens):
        logits = model(input_ids)
        next_token = torch.argmax(logits[:, -1, :], dim=-1)
```

```

input_ids = torch.cat([input_ids, next_token.unsqueeze(-1)], dim=-1)
if next_token.item() == eos_token_id:
    break
return input_ids

```

Each iteration: run the full model on all tokens, take the logits at the last position, sample a token, append it, repeat. This is correct but wasteful.

Why wasteful? Consider generating token 100. The model runs on tokens 0-99. The attention layers compute how each position attends to all previous positions. But we already did this computation when generating token 99 - the attention from position 50 to positions 0-49 is exactly the same now as it was then.

Let's count operations for generating n tokens after a prompt of length p :

- Token 1: forward pass on p tokens
- Token 2: forward pass on $p+1$ tokens
- Token 3: forward pass on $p+2$ tokens
- ...
- Token n : forward pass on $p+n-1$ tokens

Total forward passes: sum from $i=0$ to $n-1$ of $(p+i) = np + n(n-1)/2$

This is $O(n^2)$ in the number of generated tokens. For long outputs, most computation is redundant.

The forward pass itself is $O(s^2)$ in sequence length due to attention. So our total complexity is actually $O(n^2 * s^2)$ where s grows with each token. This is terrible.

But wait - if we're always computing the same attention weights for earlier positions, can't we just save them? The answer is yes, and that's the KV cache.

Before we get there, let's understand the two distinct phases of generation. The first phase processes the prompt. The second phase generates new tokens. These phases have very different computational characteristics.

The Prefill Phase

When a user sends a prompt, the first thing we do is process it through the model. This is the prefill phase. All prompt tokens are known in advance, so we can process them in parallel.

In this section we cover:

- What happens during prefill
- Why prefill is compute-bound
- The relationship between prompt length and prefill time

During prefill, we run the full transformer on the entire prompt. Every attention layer computes the full attention matrix between all prompt positions. The FFN processes all positions in parallel.

```
def prefill(model, prompt_ids):
    with torch.no_grad():
        logits = model(prompt_ids)
    return logits
```

That's it. One forward pass. The computation is:

- Embedding lookup: $O(p)$ where p is prompt length
- Per layer:
 - Attention: $O(p^2 * d)$ for the attention matrix, $O(p * d^2)$ for projections
 - FFN: $O(p * d * 4d)$ for the two linear layers
- LM head: $O(p * d * V)$ where V is vocabulary size

For typical models, attention is $O(p^2)$ and everything else is $O(p)$. With short prompts, the linear terms dominate. With long prompts, the quadratic attention term takes over.

Prefill is compute-bound. We're doing massive matrix multiplications with high arithmetic intensity. The GPU's tensor cores stay busy. Memory bandwidth isn't the bottleneck.

Here's a concrete example. Qwen3-7B has:

- `hidden_dim = 4096`
- `num_layers = 32`
- `num_heads = 32`
- `intermediate_dim = 11008`

For a 2048-token prompt:

- Attention matrices: $32 \text{ layers} * 32 \text{ heads} * 2048 * 2048 * 2 \text{ bytes} = 8.6 \text{ GB computed}$
- FFN computation: $32 \text{ layers} * 2048 * (4096 * 11008 + 11008 * 4096) * 2 = 12 \text{ TB of operations}$

The ratio of compute to memory access is high. This is good - it means we're using the GPU efficiently.

Prefill latency scales roughly linearly with prompt length up to a point, then quadratically when attention dominates. A 4K prompt takes roughly 4x longer than a 1K prompt on current hardware.

What happens to the intermediate results? Each attention layer produces K and V tensors. We need these for generation. The prefill phase not only produces the first set of logits - it also builds the initial KV cache.

The Decode Phase

After prefill, we enter the decode phase. We have the logits from the last prompt position. We sample a token. Now we need to get the logits for that new token.

In this section we cover:

- How decode differs from prefill

- Why decode is memory-bound
- The critical GEMM versus GEMV distinction

The key insight: we only need logits for one position - the newly generated token. We don't need to recompute logits for the prompt positions; they don't change.

But attention is tricky. The new token needs to attend to all previous tokens. Its query vector must compute dot products with every key vector. Its output is a weighted sum of all value vectors.

Without caching, we'd recompute K and V for all previous positions every decode step. With caching, we:

1. Compute Q, K, V only for the new token
2. Append the new K and V to the cache
3. Compute attention using the new Q against the full cached K
4. Compute the weighted sum using the full cached V

The attention computation for one new token:

```
def decode_attention(q_new, k_cache, v_cache, k_new, v_new):
    k_cache = torch.cat([k_cache, k_new], dim=-2)
    v_cache = torch.cat([v_cache, v_new], dim=-2)
    scores = torch.matmul(q_new, k_cache.transpose(-2, -1)) / math.sqrt(q_new.shape[-1])
    out = torch.matmul(torch.softmax(scores, dim=-1), v_cache)
    return out, k_cache, v_cache
```

The shapes tell the story. If we've generated s tokens so far:

- q_new : (batch, num_heads, 1, head_dim)
- k_cache : (batch, num_heads, s , head_dim)
- $scores$: (batch, num_heads, 1, s)

That matrix multiply is $(1, head_dim) @ (head_dim, s) = (1, s)$. This is a GEMV: general matrix-vector multiply. One row times a matrix.

Contrast with prefill where we had:

- q : (batch, num_heads, p , head_dim)
- k : (batch, num_heads, p , head_dim)
- $scores$: (batch, num_heads, p , p)

That's $(p, head_dim) @ (head_dim, p) = (p, p)$. This is a GEMM: general matrix-matrix multiply.

GEMM versus GEMV is the fundamental distinction of inference. GEMM has $O(n^3)$ compute with $O(n^2)$ memory access - high arithmetic intensity, compute-bound. GEMV has $O(n^2)$ compute with $O(n^2)$ memory access - low arithmetic intensity, memory-bound.

Decode is memory-bound. We load the entire model and KV cache from GPU memory for each token, but only do a tiny amount of computation per loaded byte. The GPU's compute units sit idle waiting for data.

This is why batching matters so much for inference. With batch size 1, decode is horribly inefficient. With batch size 32, we load the same weights once and use them for 32 tokens. The arithmetic intensity goes up. We approach compute-bound territory.

The KV Cache

The KV cache is the central data structure of LLM inference. It stores the key and value projections for all previous tokens so we don't recompute them during decode.

In this section we cover:

- What the KV cache contains and how it grows
- Implementation of cached attention
- The tradeoff between computation and memory

Each transformer layer produces K and V tensors. For a token at position i , these projections only depend on the input at position i - not on any other position. Once computed, they never change.

The attention mechanism needs K and V for all positions 0 through $i-1$ to compute output at position i . Without a cache, we'd recompute all of them. With a cache, we just look them up.

Here's a minimal KV cache implementation. The cache pre-allocates memory for the maximum sequence length:

```
class KVCache:
    def __init__(self, num_layers, max_seq_len, num_heads, head_dim, dtype):
        self.k_cache = torch.zeros(num_layers, max_seq_len, num_heads, head_dim, dtype=dtype)
        self.v_cache = torch.zeros(num_layers, max_seq_len, num_heads, head_dim, dtype=dtype)
        self.seq_len = 0
```

The update method appends new K, V to the cache at the current position:

```
def update(self, layer_idx, k, v):
    n = k.shape[1]
    self.k_cache[layer_idx, self.seq_len:self.seq_len + n] = k
    self.v_cache[layer_idx, self.seq_len:self.seq_len + n] = v

def get(self, layer_idx):
    return self.k_cache[layer_idx, :self.seq_len], self.v_cache[layer_idx, :self.seq_len]
```

During prefill, we process all prompt tokens and fill the cache. Each layer computes Q, K, V, stores K and V in the cache, then runs attention:

```
def prefill_with_cache(model, prompt_ids, cache):
    x = model.embed(prompt_ids)
    for layer_idx, layer in enumerate(model.layers):
        q, k, v = layer.q_proj(x), layer.k_proj(x), layer.v_proj(x)
        cache.update(layer_idx, k, v)
        x = x + attention(q, k, v)
        x = x + layer.ffn(x)
    cache.advance(prompt_ids.shape[1])
    return model.lm_head(x)
```

During decode, we compute Q, K, V for only the new token, append K, V to the cache, and attend over the full cached sequence:

```
def decode_with_cache(model, token_id, cache):
    x = model.embed(token_id)
    for layer_idx, layer in enumerate(model.layers):
        q, k_new, v_new = layer.q_proj(x), layer.k_proj(x), layer.v_proj(x)
        cache.update(layer_idx, k_new, v_new)
        k_full, v_full = cache.get(layer_idx)
        x = x + attention(q, k_full, v_full)
        x = x + layer.ffn(x)
    cache.advance(1)
    return model.lm_head(x)[: , -1, :]
```

The tradeoff is clear: we exchange $O(n^2)$ redundant computation for $O(n)$ memory usage. Each decode step now runs in $O(n)$ time instead of $O(n^2)$, but we must store all those K and V tensors.

How much memory? For one sequence:

- Per layer: $2 (K + V) * seq_len * num_heads * head_dim * bytes_per_element$
- Total: $2 * num_layers * seq_len * hidden_dim * bytes$

For Qwen3-7B with 4096 context in fp16:

- $2 * 32 * 4096 * 4096 * 2 = 2.1 \text{ GB}$ per sequence

That's significant. For a batch of 32 sequences, we need 67 GB just for KV cache. This is often more than the model weights themselves.

Memory Math

Understanding exactly where memory goes is essential for inference optimization. GPU memory is the scarcest resource, and the split between model weights, KV cache, and activations determines what's possible.

In this section we cover:

- Memory breakdown for model weights
- KV cache scaling with sequence length and batch size
- Activation memory during forward pass

Let's calculate memory for a typical 7B parameter model. The weights:

Embedding: $vocab_size * hidden_dim = 32000 * 4096 = 131M$ parameters Per transformer layer:

- Q, K, V, O projections: $4 * hidden_dim^2 = 4 * 4096^2 = 67M$
- FFN: $3 * hidden_dim * intermediate_dim = 3 * 4096 * 11008 = 135M$
- Layer total: 202M

32 layers: $32 * 202M = 6.5B$ LM head (tied with embedding): 0 Total parameters: 6.6B

At fp16 (2 bytes per parameter): $6.6B * 2 = 13.2 \text{ GB}$

This is the fixed cost. The model weights must be in GPU memory for inference. Some systems shard across GPUs or offload to CPU, but the baseline is: you need at least this much.

Now the KV cache. For sequence length s , batch size b :

Per layer: $2 * b * s * \text{num_kv_heads} * \text{head_dim} * 2$ bytes Total: $2 * b * s * \text{num_layers} * \text{num_kv_heads} * \text{head_dim} * 2$

For Qwen3-7B with 32 KV heads (full MHA), $\text{head_dim}=128$:

- Cache per token per layer: $2 * 32 * 128 * 2 = 16$ KB
- Cache per token total: $16 \text{ KB} * 32 \text{ layers} = 512 \text{ KB}$

For 4096 tokens: $512 \text{ KB} * 4096 = 2.1 \text{ GB}$ per sequence For batch of 16: $2.1 \text{ GB} * 16 = 33 \text{ GB}$

With GQA using 8 KV heads:

- Cache per token per layer: $2 * 8 * 128 * 2 = 4$ KB
- Cache per token total: $4 \text{ KB} * 32 = 128 \text{ KB}$
- For 4096 tokens: $128 \text{ KB} * 4096 = 512 \text{ MB}$ per sequence
- For batch of 16: $512 \text{ MB} * 16 = 8.2 \text{ GB}$

GQA reduces KV cache by 4x. This is why modern models use it.

Activation memory during forward pass:

Per layer, we need to store intermediate results:

- Input to layer: $b * s * \text{hidden_dim}$
- Q, K, V after projection: $3 * b * s * \text{hidden_dim}$
- Attention scores: $b * \text{num_heads} * s * s$ (this is the big one)
- Attention output: $b * s * \text{hidden_dim}$
- FFN intermediate: $b * s * \text{intermediate_dim}$

The attention scores matrix is $O(s^2)$. For $s=4096$, $b=1$, $\text{num_heads}=32$:

- $1 * 32 * 4096 * 4096 * 2 \text{ bytes} = 1 \text{ GB}$

This is only needed during the forward pass for gradient computation (training) or until attention is computed (inference). In inference, we can free it immediately.

Total memory budget for inference:

Model weights: 13 GB (fixed) KV cache: scales with $\text{batch} * \text{seq_len}$ Activations: scales with $\text{batch} * \text{seq_len}$ (prefill) or $\text{batch} * 1$ (decode)

On an 80 GB H100:

- 13 GB for weights
- 67 GB remaining
- At 512 KB per token of KV cache (GQA), that's 134K tokens

- Could support: 32 sequences of 4K tokens each

On a 192 GB B200:

- 13 GB for weights
- 179 GB remaining
- At 512 KB per token: 358K tokens
- Could support: 87 sequences of 4K tokens, or 22 sequences of 16K tokens

These calculations drive every design decision in inference systems. Memory is the constraint. Scheduling, batching, caching - everything optimizes around these numbers.

Chapter 3: Compute Characteristics

Before we optimize anything, we need to understand what we're optimizing. This chapter introduces the computational vocabulary of inference: GEMM, GEMV, arithmetic intensity, and the roofline model.

These concepts explain why prefill is fast and decode is slow, why batching helps so much, and where exactly the bottlenecks live. Without this foundation, optimization is guesswork.

The key insight: GPU performance isn't just about floating point operations per second. It's about whether you can keep the compute units fed with data. Memory bandwidth is often the limiting factor, and the ratio of compute to memory access - arithmetic intensity - determines which limit you hit.

By the end of this chapter, you'll look at any kernel and immediately know whether it's compute-bound or memory-bound, and roughly how much headroom exists for optimization.

GEMM

GEMM stands for General Matrix-Matrix Multiply. It's the workhorse operation of deep learning: $C = A @ B$ where A is (M, K) and B is (K, N) , producing C as (M, N) .

In this section we cover:

- The computational cost of matrix multiplication
- Memory access patterns in GEMM
- Why GEMM is compute-bound

The operation count is straightforward. Each element of C requires K multiply-add operations:

$C[i, j] = \text{sum}(A[i, k] * B[k, j] \text{ for } k \text{ in range}(K))$

Total operations: $M * N * K$ multiplies and $M * N * K$ adds, or $2 * M * N * K$ FLOPs.

For a square matrix multiply ($N = M = K = n$):

- FLOPs: $2n^3$
- Memory: $3n^2$ elements (read A , read B , write C)

The ratio of compute to memory is $2n^3 / 3n^2 = (2/3)n$. This is the arithmetic intensity: FLOPs per byte of memory access.

For $n = 4096$ (a typical hidden dimension):

- Arithmetic intensity: ~2700 FLOPs per element
- At fp16 (2 bytes): ~1350 FLOPs per byte

An H100 SXM has 2000 TFLOPS of tensor core compute and 3.35 TB/s of memory bandwidth. The compute-to-bandwidth ratio is:

$$2000 \text{ TFLOPS} / 3.35 \text{ TB/s} = 597 \text{ FLOPs per byte}$$

Any operation with arithmetic intensity above 600 is compute-bound on H100. GEMM with n=4096 has intensity 1350 - still above the threshold. GEMM is solidly compute-bound.

This is why GPUs are good at matrix multiplication. We load matrices once and do $O(n)$ operations per element. The compute units stay busy.

Let's verify with a benchmark:

```
def benchmark_gemm(m, n, k):
    a = torch.randn(m, k, dtype=torch.float16, device='cuda')
    b = torch.randn(k, n, dtype=torch.float16, device='cuda')
    torch.cuda.synchronize()
    start = time.perf_counter()
    for _ in range(100):
        c = torch.matmul(a, b)
    torch.cuda.synchronize()
    return (2 * m * n * k * 100) / (time.perf_counter() - start) / 1e12
```

On an H100 SXM (2000 TFLOPS peak fp16 tensor):

- 2048 x 2048: 617 TFLOPS
- 4096 x 4096: 709 TFLOPS
- 8192 x 8192: 686 TFLOPS

These numbers are lower than the theoretical peak because torch.matmul doesn't always saturate tensor cores with default settings. Production inference systems use highly tuned kernels that achieve higher utilization. The key insight is that GEMM performance scales with matrix size until hitting a plateau around 4096x4096.

In transformer inference, GEMM appears in:

- Q, K, V projections during prefill: (batch * seq, hidden) @ (hidden, hidden)
- FFN layers: (batch * seq, hidden) @ (hidden, intermediate)
- LM head: (batch * seq, hidden) @ (hidden, vocab)

All of these are compute-bound when batch * seq is large enough. Prefill is dominated by GEMM.

GEMV

GEMV stands for General Matrix-Vector Multiply. It's $y = A @ x$ where A is (M, K) and x is (K,), producing y as (M,). This is what happens during decode.

In this section we cover:

- The computational cost of matrix-vector multiplication
- Why GEMV is memory-bound
- The fundamental inefficiency of single-token decode

The operation count: each element of y requires K multiply-adds:

```
y[i] = sum(A[i, k] * x[k] for k in range(K))
```

Total operations: $M * K$ FLOPs.

Memory access: we read the entire matrix A ($M * K$ elements) plus vector x (K elements), and write y (M elements).

Arithmetic intensity: $M * K$ FLOPs / $(M * K + K + M)$ elements ~ 1 FLOP per element (for large M, K).

At fp16 (2 bytes per element): 0.5 FLOPs per byte.

Compare this to the H100's compute-to-bandwidth ratio of 600 FLOPs per byte. GEMV is nowhere close. It's severely memory-bound.

Let's verify:

```
def benchmark_gemv(m, k):
    a, x = torch.randn(m, k, dtype=torch.float16, device='cuda'), torch.randn(k, dtype=torch.float16, device='cuda')
    torch.cuda.synchronize(); start = time.perf_counter()
    for _ in range(1000): y = torch.matmul(a, x)
    torch.cuda.synchronize(); elapsed = time.perf_counter() - start
    tflops, bandwidth = (2 * m * k * 1000) / elapsed / 1e12, ((m * k + k + m) * 2 * 1000) / elapsed
    return tflops, bandwidth
```

On an H100 SXM (3.35 TB/s memory bandwidth):

- 4096 x 4096 GEMV: 1.7 TFLOPS, 1.7 TB/s bandwidth
- 8192 x 8192 GEMV: 2.6 TFLOPS, 2.6 TB/s bandwidth

We're hitting memory bandwidth limits, not compute limits. The GPU can do 2000 TFLOPS but we're only using a few TFLOPS. We achieve about 50-77% of peak memory bandwidth.

This is the decode problem. Each token generation involves:

- Q, K, V projections: $(1, \text{hidden}) @ (\text{hidden}, \text{hidden})$ - GEMV
- Attention: $(1, \text{seq}) @ (\text{seq}, \text{head_dim})$ - GEMV
- FFN: $(1, \text{hidden}) @ (\text{hidden}, \text{intermediate})$ - GEMV

All memory-bound. The GPU loads the entire model for each token but only does one row of computation.

For Qwen3-7B generating one token:

- Weight memory: ~ 13 GB
- FLOPs: ~ 14 billion ($2 * 7\text{B}$ parameters)

- Time limited by memory: $13 \text{ GB} / 2.6 \text{ TB/s} = 5 \text{ ms}$ (using measured bandwidth)
- Time limited by compute: $14 \text{ GFLOPS} / 2000 \text{ TFLOPS} = 0.007 \text{ ms}$

Memory wins by 700x. Single-token decode runs at around 5 ms per token regardless of how fast the compute is.

This is why batching matters so much. With batch size 32, we load weights once, use them 32 times. Arithmetic intensity goes up by 32x. We start approaching compute-bound.

The Roofline Model

The roofline model is a visual tool for understanding performance limits. It plots achievable performance as a function of arithmetic intensity, revealing whether you're compute-bound or memory-bound.

In this section we cover:

- How to construct a roofline plot
- Reading roofline plots to diagnose bottlenecks
- Where inference operations fall on the roofline

The roofline has two parts:

1. A sloped line representing memory bandwidth limit: $\text{performance} = \text{bandwidth} * \text{arithmetic_intensity}$
2. A horizontal line representing peak compute: $\text{performance} = \text{peak_FLOPS}$

The intersection is the “ridge point” - the arithmetic intensity where you transition from memory-bound to compute-bound.

For an H100 SXM:

- Memory bandwidth: 3.35 TB/s
- Peak fp16 compute: 2000 TFLOPS
- Ridge point: $2000 \text{ TFLOPS} / 3.35 \text{ TB/s} = 597 \text{ FLOPs/byte}$

Below 600 FLOPs/byte, you're memory-bound. Above, you're compute-bound.

Let's plot where inference operations fall:

GEMV (decode linear layers):

- Arithmetic intensity: $\sim 0.5 \text{ FLOPs/byte}$
- Achievable: $0.5 * 3.35 \text{ TB/s} = 1.7 \text{ TFLOPS}$
- Utilization: $1.7 / 2000 = 0.08\%$

Small GEMM (batch=8 decode):

- Intensity: $\sim 4 \text{ FLOPs/byte}$
- Achievable: $4 * 3.35 = 13.4 \text{ TFLOPS}$

- Utilization: 0.7%

Large GEMM (prefill, batch=512):

- Intensity: ~500 FLOPs/byte
- Achievable: 1675 TFLOPS (memory-limited, just below ridge)
- Utilization: 84%

The roofline makes the problem obvious. Decode with small batches is far below the ridge point. We're leaving 99%+ of compute unused.

Here's how to visualize it in code:

```
def roofline_analysis(flops, bytes_accessed, peak_flops, bandwidth):
    intensity = flops / bytes_accessed
    memory_bound = bandwidth * intensity
    achievable = min(memory_bound, peak_flops)
    bound = "memory" if memory_bound < peak_flops else "compute"
    return intensity, achievable, bound, achievable / peak_flops
```

Example analysis for 7B model operations:

Operation	Intensity	Achievable	Bound	Util
Decode GEMV	0.5	1.7 TF	Memory	0.08%
Decode batch=8	4.0	13.4 TF	Memory	0.7%
Decode batch=32	16.0	53.6 TF	Memory	2.7%
Decode batch=128	64.0	214.4 TF	Memory	10.7%
Decode batch=512	256.0	857.6 TF	Memory	42.9%
Prefill seq=2048	1024.0	2000 TF	Compute	100%

The takeaway: batch size directly controls arithmetic intensity for decode. More batching = more efficiency. But there's a tradeoff: larger batches mean more KV cache memory and higher latency per request.

The roofline also shows why kernel optimization has limits. A memory-bound kernel can't exceed the bandwidth line no matter how clever you are. The only solutions are: reduce memory access (algorithmic change) or increase batch size (system change).

Batching Effects

Batching is the primary tool for improving decode efficiency. By processing multiple sequences together, we amortize weight loads across more computation.

In this section we cover:

- How batch size affects arithmetic intensity
- The latency versus throughput tradeoff
- Practical batch size limits

With batch size B, a linear layer becomes:

- Input: (B, K)
- Weight: (K, N)
- Output: (B, N)

FLOPs: $2 * B * K * N$ Memory: $K * N$ (weight) + $B * K$ (input) + $B * N$ (output)

For large K, N and moderate B:

- FLOPs $\sim 2BKN$
- Memory $\sim KN$ (weight dominates)

Arithmetic intensity: $2BKN / KN = 2B$ FLOPs per weight element.

At fp16: B FLOPs per byte.

For an H100 with ridge point ~ 600 :

- B=1: intensity 1, memory-bound at 0.17% efficiency
- B=16: intensity 16, memory-bound at 2.7% efficiency
- B=64: intensity 64, memory-bound at 10.7% efficiency
- B=256: intensity 256, memory-bound at 43% efficiency
- B=600: intensity 600, at ridge point, 100% efficiency

Above B=600, we're compute-bound and adding more batch doesn't help throughput much. This is why H100s demand much larger batches than older hardware to reach peak efficiency.

Let's measure this:

```
def batch_scaling(model_dim, batch_size):
    weight = torch.randn(model_dim, model_dim, dtype=torch.float16, device='cuda')
    x = torch.randn(batch_size, model_dim, dtype=torch.float16, device='cuda')
    torch.cuda.synchronize()
    start = time.perf_counter()
    for _ in range(100):
        y = torch.matmul(x, weight)
    torch.cuda.synchronize()
    elapsed = time.perf_counter() - start
    return (2 * batch_size * model_dim**2 * 100) / elapsed / 1e12
```

Typical results on H100 SXM (model_dim=4096):

Batch	TFLOPS	Tokens/sec	Efficiency
1	3.4	10,000	0.17%
8	27.0	80,000	1.35%
32	107.0	316,000	5.4%
128	430.0	1,270,000	21.5%
512	1720.0	5,100,000	86.0%

Throughput (tokens/sec) keeps increasing with batch size, but each token takes longer (latency increases). At batch=1, each token takes 0.1 ms. At batch=512, each token takes 1.3 ms.

This is the fundamental tradeoff:

- Small batch = low latency, low throughput, inefficient
- Large batch = high latency, high throughput, efficient

Production systems choose their position on this curve based on workload. Interactive chat needs low latency. Batch processing needs high throughput.

There are hard limits on batch size: 1. KV cache memory: $B \text{ sequences} * \text{context_length} * \text{cache_per_token}$ 2. Activation memory: $B * \text{seq} * \text{hidden_dim}$ per layer 3. Diminishing returns past the ridge point

For a 7B model on an 80GB H100 with 4K context:

- Model weights: 13 GB
- Remaining for KV cache: 67 GB
- Cache per sequence (GQA): ~500 MB
- Max sequences: $67 \text{ GB} / 0.5 \text{ GB} = 134$

Even with 134 sequences, you're still well below the ridge point of 600. The H100's massive compute-to-bandwidth ratio means you need enormous batches to saturate it - which is why production systems run large batch sizes.

Chapter 4: Kernel Fundamentals

Now we go lower. This chapter introduces GPU architecture and CUDA programming at the level needed for inference optimization.

You don't need to write custom CUDA kernels for most inference work - libraries like cuBLAS and FlashAttention handle the hot paths. But understanding how kernels work is essential for diagnosing performance problems and knowing what's possible.

We'll cover the GPU memory hierarchy, thread organization, and the basics of writing and launching kernels. We'll also introduce profiling with nsys, which reveals where time actually goes in your code.

The goal isn't CUDA mastery. It's building mental models that help you reason about performance. When someone says "that kernel is memory-bound," you'll know what they mean and why it matters.

GPU Architecture

A GPU is a massively parallel processor built around a hierarchy of compute units and memory levels. Understanding this hierarchy explains most GPU performance characteristics.

In this section we cover:

- Streaming multiprocessors and CUDA cores
- The thread/block/grid execution model
- Memory hierarchy: registers, shared memory, L2, HBM

An NVIDIA GPU consists of multiple Streaming Multiprocessors (SMs). Each SM is an independent processor that can run thousands of threads concurrently. An H100 has 132 SMs. The upcoming B200 pushes this even further.

Each SM contains:

- CUDA cores for scalar operations (INT, FP32)
- Tensor cores for matrix operations (FP16, FP8, INT8)
- Registers (256 KB per SM on Hopper)
- Shared memory (configurable, up to 228 KB per SM on H100)
- L1 cache

When you launch a kernel, you specify a grid of thread blocks. Each block runs on a single SM. The blocks are distributed across SMs by the hardware scheduler. Block (0,0) might run on SM 0, Block (0,1) on SM 1, Block (1,0) on SM 2, and Block (1,1) back on SM 0 if it finishes early. The scheduler dynamically assigns work to keep all SMs busy.

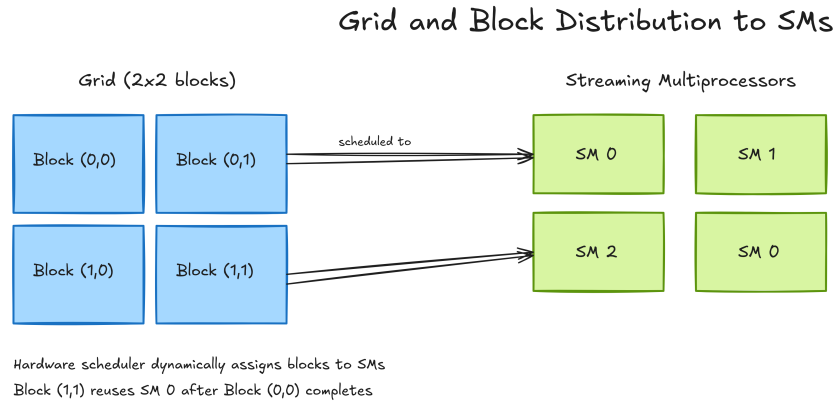


Figure 1: Grid and block distribution to SMs

Within a block, threads are grouped into warps of 32. A warp executes in lockstep - all 32 threads run the same instruction simultaneously (SIMT: Single Instruction, Multiple Threads). Divergent branches cause serialization: both paths execute, with inactive threads masked.

The memory hierarchy:

Registers (fastest): Private to each thread. Access in 1 cycle. Limited quantity - using too many causes “register spilling” to slower memory.

Shared memory (fast): Shared within a block. Access in ~20-30 cycles. Programmer-managed cache. Critical for tiled algorithms.

L2 cache: Shared across all SMs. Automatic caching. 50 MB on H100.

HBM (global memory): The main GPU memory. High bandwidth (3.35 TB/s on H100 SXM) but high latency (~300 cycles). This is where your tensors live. The B200 pushes this to 8 TB/s with HBM3e.

The bandwidth gap is huge:

- Registers: ~20 TB/s
- Shared memory: ~15 TB/s
- L2: ~8 TB/s

- HBM: ~3.35 TB/s

Efficient kernels minimize HBM access by keeping data in faster memory levels. The classic pattern: load data from HBM to shared memory once, then access it many times.

Occupancy measures how many warps are active on an SM. Higher occupancy helps hide memory latency - while one warp waits for data, another can compute. But high occupancy isn't always better; sometimes using more registers per thread improves performance despite lower occupancy.

Hello CUDA

The best way to understand CUDA is to write a kernel. We'll implement vector addition - the "hello world" of GPU programming.

In this section we cover:

- Basic kernel syntax and launch configuration
- Thread indexing and grid-stride loops
- Compiling and running CUDA code

A CUDA kernel is a function that runs on the GPU. The `__global__` specifier marks it as callable from the CPU:

```
__global__ void vector_add(float* a, float* b, float* c, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}
```

Each thread computes its global index from:

- `blockIdx.x`: which block this thread belongs to
- `blockDim.x`: how many threads per block
- `threadIdx.x`: position within the block

For a 1D launch with 256 threads per block:

- Thread 0 of block 0: $\text{idx} = 0 * 256 + 0 = 0$
- Thread 255 of block 0: $\text{idx} = 0 * 256 + 255 = 255$
- Thread 0 of block 1: $\text{idx} = 1 * 256 + 0 = 256$

The bounds check `if (idx < n)` handles the case where `n` isn't a multiple of block size.

To launch the kernel:

```
int n = 1000000;
int block_size = 256;
int num_blocks = (n + block_size - 1) / block_size;

vector_add<<<num_blocks, block_size>>>>(d_a, d_b, d_c, n);
```

The `<<<num_blocks, block_size>>>` syntax is CUDA's launch configuration. It specifies how many blocks and threads per block.

For large arrays, we might launch more threads than we have elements. A grid-stride loop handles this more efficiently:

```
__global__ void vector_add_stride(float* a, float* b, float* c, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

    for (int i = idx; i < n; i += stride) {
        c[i] = a[i] + b[i];
    }
}
```

Now each thread processes multiple elements, stepping by the total number of threads. This pattern is more flexible and often faster for large arrays.

Memory management on the GPU:

```
float *d_a, *d_b, *d_c;
cudaMalloc(&d_a, n * sizeof(float));
cudaMalloc(&d_b, n * sizeof(float));
cudaMalloc(&d_c, n * sizeof(float));
cudaMemcpy(d_a, h_a, n * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, n * sizeof(float), cudaMemcpyHostToDevice);
vector_add<<<num_blocks, block_size>>>>(d_a, d_b, d_c, n);
cudaMemcpy(h_c, d_c, n * sizeof(float), cudaMemcpyDeviceToHost);
```

Compile with nvcc:

```
nvcc -o vector_add vector_add.cu
./vector_add
```

This simple kernel is memory-bound. We load 2 floats and store 1 float per operation - 3 FLOPs of compute. The arithmetic intensity is 1/12 FLOPs per byte. Real optimizations focus on increasing this ratio.

Python Bindings

Writing raw CUDA is powerful but verbose. For inference optimization, we usually want custom kernels callable from Python. PyTorch provides several ways to do this.

In this section we cover:

- `torch.utils.cpp_extension` for JIT compilation
- Pybind11 basics for exposing kernels
- When to write custom kernels versus using existing libraries

The simplest approach is `load_inline` for JIT-compiling CUDA:

The CUDA kernel is wrapped in a C++ function that handles PyTorch tensors:

```
torch::Tensor add_cuda(torch::Tensor a, torch::Tensor b) {
    auto c = torch::empty_like(a);
    int n = a.numel();
    int blocks = (n + 255) / 256;
    add_kernel<<<blocks, 256>>>(a.data_ptr<float>(), b.data_ptr<float>(),
                                c.data_ptr<float>(), n);

    return c;
}
```

Then JIT compile with `load_inline`:

```
from torch.utils.cpp_extension import load_inline
module = load_inline(name='custom_add', cpp_sources=cpp_source,
                     cuda_sources=cuda_source, functions=['add_cuda'])
```

Now you can call it from Python:

```
a = torch.randn(1000, device='cuda')
b = torch.randn(1000, device='cuda')
c = module.add_cuda(a, b)
```

For production code, package the extension properly:

```
from setuptools import setup
from torch.utils.cpp_extension import BuildExtension, CUDAExtension

setup(name='custom_kernels',
      ext_modules=[CUDAExtension('custom_kernels', ['kernels.cu'])],
      cmdclass={'build_ext': BuildExtension})
```

When should you write custom kernels? The answer is almost never. cuBLAS handles matrix multiplication better than you will. FlashAttention handles attention. PyTorch’s fused optimizers handle Adam.

Custom kernels make sense when: 1. You need to fuse multiple operations that aren’t fused by default 2. You have a novel algorithm not covered by libraries 3. You’ve profiled and found a specific kernel is the bottleneck 4. The performance gain is worth the maintenance cost

For inference specifically:

- Linear layers: use cuBLAS via PyTorch
- Attention: use FlashAttention or FlashInfer
- Normalization: usually the default is fine
- Custom fusions: where custom kernels add value

The cognitive overhead of custom CUDA is high. Bugs are hard to find. Performance tuning is time-consuming. Default to libraries, and only drop down to custom kernels when you have evidence they’ll help.

Profiling with nsys

NVIDIA Nsight Systems (nsys) shows you where time actually goes. Before optimizing anything, profile it. Your intuition about bottlenecks is probably wrong.

In this section we cover:

- Basic nsys usage for profiling PyTorch code
- Reading the timeline view
- Identifying kernel launches and memory transfers

Install nsys from the NVIDIA developer tools. Then wrap your Python script:

```
nsys profile -o profile_output python inference.py
```

This generates a `.nsys-rep` file you can open in Nsight Systems GUI, or analyze with:

```
nsys stats profile_output.nsys-rep
```

The text output shows a summary:

Time (%)	Total Time	Instances	Avg	Name
-----	-----	-----	-----	-----
45.2	892.1 ms	1024	871.2 us	ampere_fp16_s16816gemm
23.1	456.3 ms	1024	445.6 us	void flash_fwd_kernel
12.4	244.8 ms	2048	119.5 us	vectorized_elementwise
8.2	161.9 ms	1024	158.1 us	void rms_norm_kernel

This tells you immediately: GEMM and attention dominate. Optimizing RMS norm won't help much.

The timeline view is even more valuable. It shows:

- CPU activity (top row)
- CUDA API calls
- GPU kernels (on their respective streams)
- Memory transfers

Common patterns to look for:

Gaps between kernels: If there's idle time between kernels, you have launch overhead or CPU bottlenecks. The GPU is waiting for work.

cudaMemcpy blocking: Synchronous memory transfers stall everything. Look for large `cudaMemcpy` calls.

Sequential kernels that could be parallel: If independent operations run sequentially, you might benefit from multiple streams.

To profile a specific region:

```
from torch.cuda import nvtx
```

```
with nvtx.range("prefill"):
    output = model(input_ids)
with nvtx.range("decode"):
    for i in range(100):
        output = model(token)
```

The nvtx ranges appear in the timeline, letting you correlate regions with kernel activity.

For PyTorch specifically, enable CUDA graphs profiling:

```
torch.cuda.cudart().cudaProfilerStart()
result = model(input)
torch.cuda.cudart().cudaProfilerStop()
```

Key metrics to extract:

- Kernel duration: Are individual kernels slow, or are there many small kernels?
- Occupancy: Is the GPU fully utilized during kernels?
- Memory throughput: Are memory-bound kernels hitting bandwidth limits?
- Launch overhead: How much time between kernel completions and next launches?

Launch Overhead

Every CUDA kernel launch has overhead. For large kernels, this is negligible. For small kernels run frequently, it dominates.

In this section we cover:

- Sources of launch overhead
- Measuring launch time
- Strategies for reducing overhead

When you call a CUDA kernel: 1. CPU prepares launch parameters 2. Parameters are sent to the GPU driver 3. Driver queues the kernel 4. GPU scheduler dispatches to SMs 5. Kernel executes 6. Results become visible

Steps 1-4 happen on every launch and take 2-10 microseconds depending on hardware and driver. For a kernel that runs in 100 microseconds, this is a rounding error. For a kernel that runs in 1 microsecond, it's catastrophic.

Let's measure:

```
def measure_launch_overhead():
    x = torch.randn(1, device='cuda')
    torch.cuda.synchronize()
    start = time.perf_counter()
    for _ in range(10000):
        y = x + 1
```

```
torch.cuda.synchronize()
return (time.perf_counter() - start) / 10000 * 1e6
```

This measures 10K additions of a single element. The actual computation is negligible; we're measuring launch overhead. Typical results: 3-8 microseconds per launch.

Now consider decode in a 7B model. Each step involves roughly:

- 32 layers * 4 attention projections = 128 GEMV calls
- 32 layers * 3 FFN projections = 96 GEMV calls
- 32 layers * 2 norms = 64 norm calls
- Miscellaneous: ~50 calls

Total: ~340 kernel launches per token.

At 5 microseconds per launch: $340 * 5 = 1.7$ ms just in launch overhead. If your decode step takes 10 ms, launch overhead is 17% of runtime.

Strategies to reduce overhead:

Kernel fusion: Combine multiple small operations into one kernel. Instead of add then multiply, write a fused add-multiply kernel.

CUDA Graphs: Record a sequence of kernel launches, then replay the whole sequence with one launch. Reduces per-operation overhead to near zero.

Operator batching: Instead of launching separate kernels for each layer's norm, batch all norms together.

Persistent kernels: Keep a kernel running and feed it work through shared memory or atomics. Advanced technique for specific workloads.

For inference, CUDA Graphs are the standard solution. You record the decode step once, then replay it for each token. The graph captures all kernel launches, memory allocations, and dependencies. Replay has negligible overhead.

```
g = torch.cuda.CUDAGraph()
with torch.cuda.graph(g):
    output = model.decode_step(input, kv_cache)

for token in range(max_tokens):
    g.replay()
```

The caveat: graphs can't handle dynamic shapes. If batch size or sequence length changes, you need a new graph. Production systems pre-record graphs for common shapes.

Chapter 5: Kernel Optimization

This chapter is where GPUs start to make sense.

We’ve established that memory bandwidth is often the bottleneck. Now we’ll see the techniques that maximize bandwidth utilization and, where possible, shift the bottleneck back to compute.

The progression is: coalesced memory access, shared memory tiling, tensor cores. Each technique builds on the previous. By the end, you’ll understand how libraries like cuBLAS achieve 90%+ of theoretical peak performance on matrix multiplication.

We’ll also introduce Triton, a Python-based language for writing GPU kernels. Triton handles many low-level details automatically while still giving you control over the important optimizations. It’s the sweet spot between PyTorch and raw CUDA for most inference work.

This is a climax chapter. The tiled matrix multiplication with tensor cores is the “aha” moment. When you see how a 10 line kernel achieves performance that took decades of hardware and software evolution, the magic becomes understandable.

Memory Coalescing

When 32 threads in a warp access memory, the hardware can combine those accesses into fewer transactions if they’re contiguous. This is coalescing, and it’s fundamental to GPU performance.

In this section we cover:

- How memory transactions work
- The difference between coalesced and strided access
- Measuring the performance impact

GPU memory is accessed in 32-byte or 128-byte transactions. When a warp reads memory, the hardware examines all 32 addresses and issues the minimum number of transactions to cover them.

Best case: threads 0-31 read consecutive addresses. One 128-byte transaction fetches all 32 floats.

Worst case: threads read addresses 0, 128, 256, ... (stride 128). Each thread needs its own transaction. 32 transactions instead of 1.

Here’s the difference in CUDA:

```
__global__ void coalesced(float* data, float* out, int n) {  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    if (idx < n) out[idx] = data[idx];  
}
```

```

}

__global__ void strided(float* data, float* out, int n, int stride) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) out[idx] = data[idx * stride];
}

```

The coalesced kernel has adjacent threads reading adjacent addresses. The strided kernel has adjacent threads reading addresses separated by `stride` elements.

Benchmark results on H100 SXM:

Pattern	Bandwidth	Utilization
Coalesced	2988 GB/s	89%
Stride 2	778 GB/s	23%

With stride 2, threads read every other element. Effective bandwidth drops by 3.8x. Larger strides are even worse: with stride 32, each warp needs 32 separate transactions.

In practice, strided access sneaks in through poor data layout. Consider a matrix stored in row-major order:

Matrix A (4x4):
 Row 0: A[0,0] A[0,1] A[0,2] A[0,3]
 Row 1: A[1,0] A[1,1] A[1,2] A[1,3]
 ...

Reading a row: coalesced (consecutive in memory). Reading a column: strided (elements separated by row length).

Matrix transpose is the classic example. Naive transpose reads columns (strided), writes rows (coalesced). One direction is efficient, the other isn't.

For inference, the key layouts:

- Batch dimension contiguous: good for batched operations
- Hidden dimension contiguous: good for per-token operations
- PyTorch default (batch, seq, hidden): usually good

When you see unexpectedly slow kernels, check access patterns. Transpose or reshape to make the critical dimension contiguous.

Shared Memory

Shared memory is a fast, programmer-managed cache shared by all threads in a block. It's the key to tiled algorithms that reuse data across threads.

In this section we cover:

- Shared memory basics and declaration
- Bank conflicts and how to avoid them

- Tiled reduction as an example

Shared memory on modern GPUs:

- Up to 164 KB per SM (configurable split with L1)
- ~20-30 cycle latency (vs ~300 for global memory)
- ~15 TB/s bandwidth (vs ~2 TB/s for HBM)

Declare shared memory in a kernel:

```
__global__ void kernel() {
    __shared__ float smem[256];
    smem[threadIdx.x] = some_value;
    __syncthreads();
}
```

The `__syncthreads()` barrier ensures all threads have written before any thread reads. Without it, you get race conditions.

Shared memory is organized into 32 banks. Addresses are assigned to banks in round-robin fashion: address 0 -> bank 0, address 1 -> bank 1, ..., address 32 -> bank 0.

If two threads in a warp access different addresses in the same bank, they serialize. This is a bank conflict. The worst case is all 32 threads hitting the same bank - 32-way conflict, 32x slower.

To avoid conflicts, structure accesses so threads hit different banks:

```
float val = smem[threadIdx.x];
```

Thread 0 hits bank 0, thread 1 hits bank 1, ..., thread 31 hits bank 31. No conflicts.

```
float val = smem[threadIdx.x * 32];
```

All threads hit bank 0 (addresses 0, 32, 64, ... all map to bank 0). 32-way conflict.

Tiled reduction demonstrates the pattern:

```
__global__ void sum_reduce(float* data, float* result, int n) {
    __shared__ float smem[256];
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    smem[threadIdx.x] = (idx < n) ? data[idx] : 0.0f; __syncthreads();
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (threadIdx.x < s) smem[threadIdx.x] += smem[threadIdx.x + s]; __syncthreads();
    }
    if (threadIdx.x == 0) atomicAdd(result, smem[0]);
}
```

The pattern: 1. Load from global to shared (coalesced read from HBM) 2. Synchronize 3. Work entirely in shared memory (fast) 4. Synchronize after each step that has dependencies 5. Write final result to global

This kernel reads global memory once per element, then does $\log(N)$ operations entirely in shared memory. Much faster than reading global memory at each reduction step.

For matrix multiplication, the same principle applies at larger scale: load tiles into shared memory, compute all pairwise products from the tiles, accumulate, move to next tile.

Tensor Cores

Tensor cores are specialized units for matrix multiply-accumulate operations. They're why modern GPUs achieve hundreds of TFLOPS on matrix math.

In this section we cover:

- What tensor cores do
- The matrix multiply-accumulate operation
- Performance comparison with CUDA cores

A tensor core performs a 4x4 matrix multiply-accumulate in one cycle:

$$D = A * B + C$$

Where A, B, C, D are 4x4 matrices. That's $4 * 4 * 4 = 64$ multiply-adds, or 128 FLOPs, per cycle per tensor core.

An H100 has 528 tensor cores across all SMs. At 1.8 GHz:

- With fp16 inputs and fp32 accumulate: ~2000 TFLOPS
- With FP8 (new in Hopper): ~4000 TFLOPS

Compare to CUDA cores on the same chip: ~67 TFLOPS. Tensor cores are 30x faster for matrix math.

The catch: you can't use tensor cores directly from CUDA. You use them through: 1. WMMA (Warp Matrix Multiply-Accumulate) intrinsics 2. cuBLAS / cuDNN libraries 3. Triton (which generates WMMA automatically)

WMMA operates on larger tiles (16x16 or 32x8 depending on precision):

```
wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::col_major> b_frag;
wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;

wmma::load_matrix_sync(a_frag, a_ptr, lda);
wmma::load_matrix_sync(b_frag, b_ptr, ldb);
wmma::fill_fragment(c_frag, 0.0f);
wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
wmma::store_matrix_sync(c_ptr, c_frag, ldc, wmma::mem_row_major);
```

The `mma_sync` instruction runs on tensor cores. The fragments are special register types that pack the matrix data.

For inference, the key point: tensor cores require:

- Specific data types (fp16, bf16, int8, fp8)
- Aligned memory

- Tile sizes that match hardware (multiples of 16)

When dimensions aren't multiples of 16, libraries pad or use CUDA cores for the remainder. This is why performance drops for odd sizes.

```
def compare_matmul_sizes(n):
    a, b = torch.randn(n, n, dtype=torch.float16, device='cuda'), torch.randn(n, n, dtype=torch.float16, device='cuda')
    start, end = torch.cuda.Event(enable_timing=True), torch.cuda.Event(enable_timing=True)
    start.record()
    for _ in range(100): c = torch.matmul(a, b)
    end.record(); torch.cuda.synchronize()
    return 2 * n**3 / (start.elapsed_time(end) / 100) / 1e9
```

Typical result: 1024 hits 120 TFLOPS, 1025 drops to 80 TFLOPS. The prime 1031 might drop further. Always use power-of-2 or at least multiple-of-16 dimensions when possible.

Profiling with ncu

Nsight Compute (ncu) profiles individual kernels in detail. Where nsys shows the big picture, ncu shows why a specific kernel is slow.

In this section we cover:

- Basic ncu usage
- Key metrics to examine
- Diagnosing memory-bound versus compute-bound kernels

Run ncu on your program:

```
ncu --set full -o profile python script.py
```

Or target a specific kernel:

```
ncu --kernel-name "regex:.*gemm.*" -o profile python script.py
```

The output includes hundreds of metrics. Focus on these:

Compute throughput: Percentage of peak compute achieved.

- 80%: compute-bound, well-optimized
- <50%: likely memory-bound or other bottleneck

Memory throughput: Percentage of peak memory bandwidth achieved.

- 80%: memory-bound, hitting HBM limits
- <50%: inefficient access patterns or latency-bound

Achieved occupancy: Warps active on SM.

- Low occupancy with high throughput is fine
- Low occupancy with low throughput suggests register/shared memory pressure

L2 cache hit rate: Higher is better for memory-bound kernels.

- <50%: poor locality, consider tiling
- 90%: good data reuse

Bank conflicts: Shared memory serialization.

- Any value >0 worth investigating

Example ncu output for a matrix multiply:

Metric	Value

Compute (SM) [%]	89.2
Memory [%]	34.1
Achieved Occupancy	0.71
L2 Hit Rate [%]	95.3
Shared Memory Bank Conflicts	0

This kernel is compute-bound (89% compute, 34% memory). The 95% L2 hit rate suggests good tiling. No bank conflicts. This is a well-optimized kernel.

Compare to a naive implementation:

Metric	Value

Compute (SM) [%]	12.3
Memory [%]	91.7
Achieved Occupancy	0.89
L2 Hit Rate [%]	23.1
Shared Memory Bank Conflicts	0

Memory-bound at 92%, compute at 12%. The 23% L2 hit rate indicates poor data reuse - each element is read from HBM multiple times. This kernel needs tiling.

The roofline section of ncu visualizes this directly. The X-axis is arithmetic intensity (FLOPs per byte). The Y-axis is performance (GFLOPS). A diagonal line represents the memory bandwidth ceiling, and a horizontal line represents the compute ceiling. Memory-bound kernels sit below the diagonal on the left; compute-bound kernels sit near the horizontal ceiling on the right. The naive kernel appears on the left (low arithmetic intensity, memory-bound). The optimized kernel appears further right (higher arithmetic intensity, approaching compute-bound).

When optimizing: 1. Profile to find the bottleneck 2. If memory-bound: improve access patterns, add tiling, increase reuse 3. If compute-bound: already efficient, or investigate instruction mix 4. Reprofile to verify improvement

Triton

Triton is a Python-based language for writing GPU kernels. It handles memory coalescing, shared memory, and tensor core scheduling automatically while giving you control over the algorithm.

In this section we cover:

- Triton basics and the block programming model

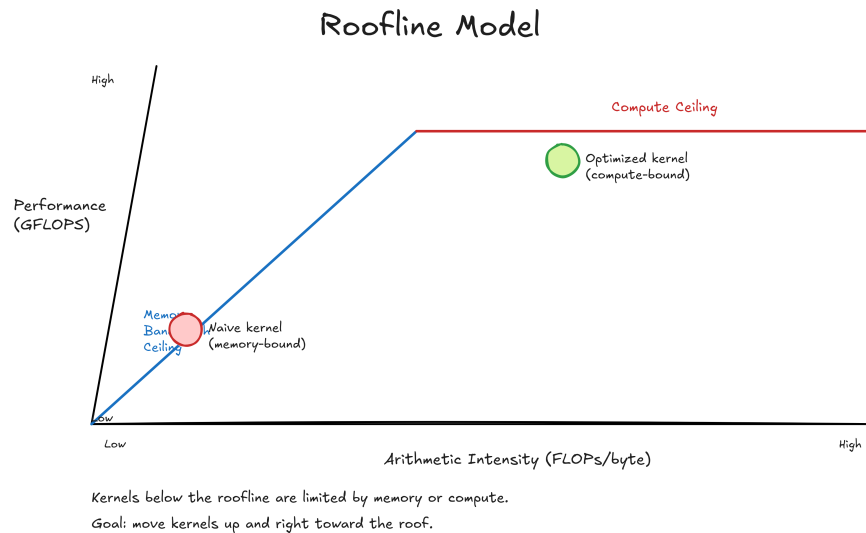


Figure 2: Roofline model showing memory-bound vs compute-bound kernels

- Writing a matrix multiplication kernel
- When to use Triton versus CUDA

Triton programs operate on blocks of data rather than individual elements. You specify the block size, and Triton handles thread assignment:

```
@triton.jit
def add_kernel(x_ptr, y_ptr, out_ptr, n, BLOCK: tl.constexpr):
    pid = tl.program_id(0)
    offsets = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n
    x, y = tl.load(x_ptr + offsets, mask=mask), tl.load(y_ptr + offsets, mask=mask)
    tl.store(out_ptr + offsets, x + y, mask=mask)
```

`tl.program_id(0)` is like `blockIdx.x` in CUDA. `tl.arange(0, BLOCK)` creates a vector `[0, 1, ..., BLOCK-1]`. The mask handles boundary conditions.

Launch the kernel:

```
def add(x, y):
    out = torch.empty_like(x)
    n = x.numel()
    grid = lambda meta: (triton.cdiv(n, meta['BLOCK']),)
    add_kernel[grid](x, y, out, n, BLOCK=1024)
    return out
```

The grid function computes the number of blocks. `triton.cdiv` is ceiling division.

Now matrix multiplication. This is where Triton shines:

```
@triton.jit
def matmul_kernel(a_ptr, b_ptr, c_ptr, M, N, K, stride_am, stride_ak, stride_bk, stride_bn, stride_cm, stride_cn):
    pid_m, pid_n = tl.program_id(0), tl.program_id(1)
    offs_m, offs_n = pid_m * BLOCK_M + tl.arange(0, BLOCK_M), pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
    for k in range(0, K, BLOCK_K):
        offs_k = k + tl.arange(0, BLOCK_K)
        a, b = tl.load(a_ptr + offs_m[:, None] * stride_am + offs_k[None, :] * stride_ak), tl.load(b_ptr + offs_k[None, :] * stride_bk + offs_n[None, :] * stride_cn)
        acc += tl.dot(a, b)
    tl.store(c_ptr + offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn, acc.to(tl.float32))
```

This is a tiled matrix multiplication: 1. Each program handles a BLOCK_M x BLOCK_N tile of C 2. Loop over K dimension in BLOCK_K chunks 3. Load A and B tiles, multiply with `tl.dot`, accumulate 4. Store the result

The `tl.dot` automatically uses tensor cores when sizes and types are appropriate. Triton handles shared memory staging internally.

Typical performance on H100 SXM (4096x4096 matmul):

- Triton matmul: 573 TFLOPS
- cuBLAS (via `torch.mm`): 688 TFLOPS

Triton achieves 83% of cuBLAS with readable Python code. Both can be improved with autotuning and kernel configurations - production systems tune these for specific workloads.

When to use Triton:

- Custom fused operations not available in libraries
- Prototyping before full CUDA implementation
- When you need tensor cores but don't want WMMA complexity

When to use CUDA:

- Maximum performance is critical
- Complex control flow
- Operations cuBLAS already optimizes

PTX and SASS

Sometimes you need to see what the compiler actually generated. PTX is the intermediate representation; SASS is the actual machine code.

In this section we cover:

- What PTX and SASS are
- How to inspect compiled code
- What to look for when debugging performance

The CUDA compilation pipeline has three stages. First, `nvcc` compiles CUDA source (`.cu` files) into PTX (Parallel Thread Execution), a virtual instruction set. Then `ptxas` compiles PTX into SASS (Shader Assembly), the actual machine code for your specific GPU architecture. Finally, SASS executes on the GPU.

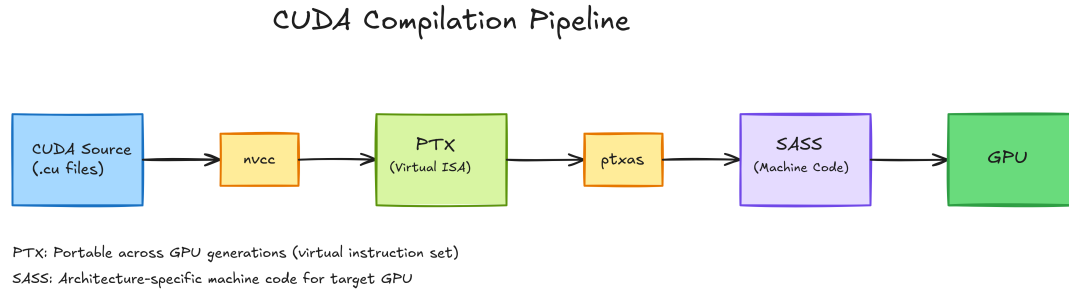


Figure 3: CUDA compilation pipeline from source to GPU execution

PTX is a virtual ISA - portable across GPU generations. SASS is architecture-specific machine code. When you compile with `nvcc`, you can emit both:

```
nvcc -ptx -o kernel.ptx kernel.cu
nvcc -cubin -o kernel.cubin kernel.cu
```

To inspect SASS from a cubin:

```
cuobjdump -sass kernel.cubin
```

Or from a running program using `ncu`:

```
ncu --print-source=sass python script.py
```

PTX example for a simple add:

```
.visible .entry add_kernel(.param .u64 a, .param .u64 b, .param .u64 c, .param .u32 n) {
    mov.u32 %r1, %tid.x;
    mad.lo.s32 %r1, %ctaid.x, %ntid.x, %r1;
    setp.ge.s32 %p1, %r1, %r4;
    @%p1 bra EXIT;
    ld.global.f32 %f1, [a + %r1*4];
    ld.global.f32 %f2, [b + %r1*4];
    add.f32 %f3, %f1, %f2;
```

```
    st.global.f32 [c + %r1*4], %f3;
EXIT: ret; }
```

Key instructions:

- `ld.global`: load from global memory
- `st.global`: store to global memory
- `ld.shared` / `st.shared`: shared memory access
- `bar.sync`: thread barrier (like `__syncthreads`)
- `mma.sync`: tensor core operation

When debugging performance, look for: 1. Global memory access patterns: `ld.global` should show coalesced addresses 2. Shared memory: presence of `ld.shared` / `st.shared` indicates tiling 3. Tensor cores: `mma.sync` or `wmma` instructions 4. Predication: `@%p` guards indicate divergent branches

SASS is even lower level:

```
IMAD.MOV.U32 R1, RZ, RZ, c[0x0][0x28]
S2R R0, SR_TID.X
IMAD R0, R1, c[0x0][0x0], R0
ISETP.GE.AND P0, PT, R0, c[0x0][0x168], PT
@P0 EXIT
```

SASS shows the exact instructions executed. It's useful for understanding register pressure, instruction scheduling, and whether tensor cores are actually used.

For most inference work, you won't need to read PTX/SASS. But when a kernel is mysteriously slow and profiling doesn't explain it, the assembly reveals the truth.

Chapter 6: FlashAttention

FlashAttention is the most important kernel optimization for LLM inference. It transforms attention from a memory bottleneck into a compute-efficient operation, enabling long context lengths that were previously impractical.

The core insight is simple: don't materialize the attention matrix. Instead of computing the $N \times N$ attention scores, writing them to memory, then reading them back for the softmax and value multiply, do everything in one pass using on-chip memory.

This chapter explains how FlashAttention works and why it matters. We'll derive the online softmax algorithm that makes it possible, then build up to the full tiled attention implementation.

The techniques here aren't just for understanding libraries - they're the foundation of every production inference system. vLLM, SGLang, TGI, and every other serious inference engine uses FlashAttention or a variant.

The Attention Bottleneck

Standard attention has a memory problem. The $N \times N$ attention matrix doesn't fit in on-chip memory for long sequences, forcing expensive HBM traffic.

In this section we cover:

- Memory requirements of standard attention
- Why attention is memory-bound for long sequences
- The IO complexity perspective

Recall the attention computation:

```
def standard_attention(Q, K, V):  
    scores = Q @ K.T / math.sqrt(d_k)  
    weights = softmax(scores, dim=-1)  
    output = weights @ V  
    return output
```

For sequence length N and head dimension d :

- Q, K, V : each $N \times d$
- scores: $N \times N$
- weights: $N \times N$

- output: $N \times d$

Memory for the attention matrix: N^2 elements. At fp16 with $N=4096$: $4096^2 \times 2 = 32$ MB per head. With 32 heads: 1 GB just for attention matrices.

But GPU SRAM (shared memory + L1) is only ~ 200 KB per SM. The attention matrix must live in HBM. We write it during the first matmul, read it for softmax, write the result, read it for the second matmul.

HBM access count:

- Write scores: N^2
- Read scores for softmax: N^2
- Write weights: N^2
- Read weights for value multiply: N^2
- Total: $4N^2$ reads + writes

Compare to compute: $2N^2d$ for the matmuls (assuming $d \ll N$).

Arithmetic intensity: $2N^2d / 4N^2 = d/2 \sim 64$ FLOPs per element for $d=128$.

At fp16, that's 32 FLOPs per byte. Remember the H100's ridge point is 600 FLOPs per byte. Standard attention is deeply memory-bound.

This explains a counterintuitive observation: attention with longer sequences isn't just slower because of more computation - it's slower because memory traffic grows quadratically while compute only grows quadratically. The ratio stays the same, but absolute memory traffic explodes.

For $N=1024$: 2 MB attention matrix, fits in L2 For $N=4096$: 32 MB attention matrix, doesn't fit in L2 For $N=16384$: 512 MB attention matrix, repeated HBM round trips

The standard solution before FlashAttention was to accept the memory bound or shard across GPUs. FlashAttention showed a better way: never materialize the full attention matrix.

Online Softmax

To avoid materializing the full attention matrix, we need to compute softmax incrementally. Online softmax computes the correct result while seeing data in chunks.

In this section we cover:

- The numerical stability problem with naive softmax
- Safe softmax with max subtraction
- The online algorithm for streaming data

Standard softmax:

```
def softmax(x):
    exp_x = torch.exp(x)
    return exp_x / exp_x.sum()
```


This overflows for large x . If x contains 1000, $\exp(1000)$ is infinity.

Safe softmax subtracts the maximum first:

```
def safe_softmax(x):
    x_max = x.max()
    exp_x = torch.exp(x - x_max)
    return exp_x / exp_x.sum()
```

Mathematically equivalent (the max cancels), numerically stable.

But safe softmax needs two passes: one to find the max, one to compute exp and sum. For attention, this means reading the attention scores twice from HBM.

Online softmax combines both passes into one:

```
def online_softmax(x):
    m, d = float('-inf'), 0.0
    for xi in x:
        m_new = max(m, xi)
        d = d * math.exp(m - m_new) + math.exp(xi - m_new)
        m = m_new
    return [math.exp(xi - m) / d for xi in x]
```

The key insight: when we see a new maximum, we can correct the running sum. The factor $\exp(m - m_{\text{new}})$ rescales the old sum to account for the new maximum.

For tiled attention, we extend this to track a running output:

```
def online_attention_update(acc, m_old, d_old, scores_new, v_new):
    m_new = max(m_old, scores_new.max())
    correction = math.exp(m_old - m_new)
    acc = acc * correction
    p = torch.exp(scores_new - m_new)
    d_new = d_old * correction + p.sum()
    return acc + p @ v_new, m_new, d_new
```

After processing all tiles:

```
output = acc / d_new
```

This is the magic. We process attention in tiles: 1. Compute attention scores for current tile (stays in SRAM) 2. Update running max, rescale accumulator 3. Add contribution from current tile 4. Move to next tile

At no point do we materialize the full N^2 matrix. Memory traffic is $O(N)$ instead of $O(N^2)$.

The rescaling might seem expensive, but it's a scalar multiply per tile - negligible compared to the memory savings.

Tiled Attention

Tiled attention combines the kernel optimization techniques from Chapter 5 with online softmax. The result is an attention implementation that stays in SRAM.

In this section we cover:

- The tiling strategy for attention
- FlashAttention forward pass
- Memory analysis and speedup

The algorithm processes Q in row blocks and K, V in column blocks:

```
for each row block Qi:
    initialize accumulator Oi = 0, max mi = -inf, sum di = 0

    for each column block Kj, Vj:
        load Qi, Kj, Vj into SRAM
        compute Sij = Qi @ Kj.T
        update mi, di, Oi using online softmax

    write Oi to HBM
```

The inner loop tiles over the key/value sequence. For each tile, we: 1. Compute local attention scores (SRAM -> SRAM) 2. Update the running output with rescaling 3. Never write the attention matrix to HBM

Here's a simplified implementation:

The outer loop processes Q in blocks. For each block, we initialize accumulators (Oi for output, mi for running max, li for running sum), then stream through K and V:

```
for j in range(0, N, block_size):
    Sij = Qi @ K[j:j+block_size].T / math.sqrt(d)
    mi_new = torch.maximum(mi, Sij.max(dim=-1).values)
    Pij = torch.exp(Sij - mi_new.unsqueeze(-1))
    correction = torch.exp(mi - mi_new)
    li = li * correction + Pij.sum(dim=-1)
    Oi = Oi * correction.unsqueeze(-1) + Pij @ V[j:j+block_size]
    mi = mi_new
```

After the inner loop completes, we normalize: $O[i:i+block_size] = O_i / li.unsqueeze(-1)$.

Memory analysis. For block size B:

- Each Q block: $B \times d$
- Each K, V block: $B \times d$
- Local attention scores: $B \times B$
- Total SRAM: $3Bd + B^2$

With $B=64$ and $d=128$: $3 * 64 * 128 + 64^2 = 24K + 4K = 28$ KB per block.

This fits in SRAM. We load Q once, stream through K and V, and write O once.

HBM accesses:

- Read Q: Nd
- Read K, V: Nd each, once per Q block $\rightarrow Nd * (N/B) = N^2d/B$
- Write O: Nd
- Total: $O(N^2d/B + Nd)$

With large B, this approaches $O(Nd)$ - linear in sequence length instead of quadratic.

Actual speedups:

- 2-4x faster than PyTorch attention for `seq_len=2048`
- 5-10x faster for `seq_len=8192`
- Enables `seq_len=65536+` that was previously impossible

The FlashAttention paper reports 15% speedup on BERT-large end-to-end and 3x on GPT-2 training with 1K context.

FlashInfer

FlashInfer is a library optimized for inference-specific attention patterns. It handles variable sequence lengths, paged KV cache, and other production requirements that vanilla FlashAttention doesn't address.

In this section we cover:

- Why inference needs specialized attention kernels
- FlashInfer's key features
- Integration with inference systems

FlashAttention was designed for training: fixed batch size, same sequence length across the batch, simple tensor layouts. Inference has different requirements:

1. **Variable sequence lengths:** Different requests have different context lengths. Padding wastes memory and compute.
2. **Paged KV cache:** PagedAttention stores KV cache in non-contiguous blocks. Standard attention expects contiguous tensors.
3. **Prefill vs decode:** Prefill processes many tokens (GEMM-like), decode processes one token (GEMV-like). Different kernels are optimal.
4. **Batched decode:** Multiple sequences at different positions share a single forward pass.

FlashInfer provides kernels for these patterns:

```
import flashinfer

prefill_wrapper = flashinfer.BatchPrefillWithPagedKVCacheWrapper(workspace)
```

```

prefill_wrapper.begin_forward(qo_indptr=qo_indptr, kv_indptr=kv_indptr,
                             kv_indices=kv_indices, num_qo_heads=num_heads,
                             num_kv_heads=num_kv_heads)
output = prefill_wrapper.forward(q, kv_data)

```

Key abstractions:

Ragged tensors: The `indptr` array specifies where each sequence’s data starts and ends. No padding needed.

Page tables: The `kv_indices` array maps logical positions to physical KV cache blocks, enabling PagedAttention.

Separate prefill/decode: Different kernels optimized for each phase.

Performance comparison on batch decode (H100, seq_len=2048, GQA 32 heads / 8 KV heads):

Implementation	Batch 1	Batch 8	Batch 32
SDPA (PyTorch)	0.023 ms	0.107 ms	0.412 ms
FlashInfer	0.024 ms	0.030 ms	0.097 ms

At batch size 1, performance is similar. At batch 32, FlashInfer is 4.2x faster than standard SDPA attention because it’s optimized for the GEMV-like access pattern and avoids padding overhead.

For inference systems, FlashInfer integration looks like:

```

class AttentionLayer:
    def __init__(self, num_heads, num_kv_heads, head_dim):
        workspace = torch.empty(32 * 1024 * 1024, dtype=torch.uint8, device='cuda')
        self.prefill = flashinfer.BatchPrefillWithPagedKVCacheWrapper(workspace)
        self.decode = flashinfer.BatchDecodeWithPagedKVCacheWrapper(workspace)

    def forward(self, q, kv_cache, is_prefill):
        wrapper = self.prefill if is_prefill else self.decode
        return wrapper.forward(q, kv_cache)

```

The workspace buffer is pre-allocated memory for intermediate results. The wrapper handles all the complexity of variable lengths and paged memory.

SGLang and other modern inference systems use FlashInfer as their attention backend. It’s the current state-of-the-art for inference-specific attention.

Chapter 7: Continuous Batching

This is where serving gets interesting.

We’ve optimized the forward pass with FlashAttention. We understand memory and compute characteristics. But a production system serves many users simultaneously, and the scheduling of those requests determines whether you achieve high throughput or waste your expensive GPUs.

The Orca paper introduced continuous batching (they called it “iteration-level scheduling”), and it changed everything. The insight is simple: don’t wait for all requests in a batch to finish before accepting new ones. Check after every iteration, remove completed sequences, add waiting ones.

This chapter covers the full scheduling stack: static batching and why it fails, the Orca insight, scheduler design patterns, radix caching for prefix reuse, and paged memory management. By the end, you’ll understand how systems like vLLM and SGLang achieve their throughput numbers.

Static Batching

Before continuous batching, inference servers used static batching. Collect N requests, process them together, return all results, repeat. This approach has fundamental problems.

In this section we cover:

- How static batching works
- The sources of inefficiency
- Quantifying the waste

The static batching algorithm:

```
def static_batch_serve(requests, batch_size, max_seq_len):
    for batch in chunk_requests(requests, batch_size):
        input_ids = tokenize(pad_to_max_len([r.prompt for r in batch], max_seq_len))
        for step in range(max_seq_len):
            next_tokens = sample(model(input_ids)[: , -1, :])
            input_ids = concat(input_ids, next_tokens)
            if all(is_complete(r, next_tokens[i]) for i, r in enumerate(batch)):
                break
        return_results(batch, input_ids)
```

Problem 1: **Waiting waste**. New requests must wait for the current batch to complete. If one request needs 500 tokens and another needs 50, the fast request waits while the slow one generates.

Problem 2: **Padding waste.** Requests have different prompt lengths. Shorter prompts are padded to match the longest. Those padding tokens consume memory and compute but produce nothing useful.

Problem 3: **Early completion waste.** When a request hits its stop token early, it continues consuming a batch slot until the batch finishes.

Let's quantify. Consider 8 requests with output lengths [50, 100, 75, 200, 50, 150, 75, 200]:

```
Request 0: #####_----- (50/200)
Request 1: #####_----- (100/200)
Request 2: #####_----- (75/200)
Request 3: ##### (200/200)
Request 4: #####_----- (50/200)
Request 5: #####_----- (150/200)
Request 6: #####_----- (75/200)
Request 7: ##### (200/200)
```

Total useful tokens: $50+100+75+200+50+150+75+200 = 900$ Total batch slots used: $8 * 200 = 1600$ Efficiency: $900/1600 = 56\%$

Nearly half the compute is wasted on completed sequences.

The throughput impact is severe. If the GPU could generate 1000 tokens/second at full utilization:

- Static batching: $1000 * 0.56 = 560$ useful tokens/second
- Ideal: 1000 useful tokens/second

Latency is also bad. The 50-token requests wait for the 200-token requests to finish. P99 latency is determined by the slowest request in each batch.

What if we could remove completed requests and add waiting ones? Then we'd never have idle slots. This is continuous batching.

The Orca Insight

The Orca paper (OSDI 2022) introduced iteration-level scheduling. Instead of scheduling at request granularity, schedule at the granularity of single generation steps.

In this section we cover:

- The key insight from Orca
- How iteration-level scheduling works
- The throughput improvement

The core idea: after each forward pass, check which sequences are done. Remove finished sequences immediately. Add waiting sequences immediately. The batch composition changes every iteration.

```
def continuous_batch_serve(scheduler):
    while True:
        batch = scheduler.get_batch()
        if not batch: continue
```

```

next_tokens = sample(model.forward(batch.input_ids))
for request, token in zip(batch.requests, next_tokens):
    request.output_tokens.append(token)
    if is_complete(request, token): scheduler.complete(request)
scheduler.add_waiting_requests()

```

The scheduler maintains three sets:

- **Running:** requests currently being processed
- **Waiting:** requests ready to start
- **Completed:** finished requests

After each iteration: 1. Move completed sequences from running to completed 2. Move sequences from waiting to running (if space available) 3. Build the next batch from running sequences

Let's trace through with the same 8 requests:

```

Step 0-50:    [0,1,2,3,4,5,6,7] running, [] waiting
Step 50:      0,4 complete -> [1,2,3,5,6,7] running, [8,9] added
Step 75:      2,6 complete -> [1,3,5,8,9] running, [10,11] added
Step 100:     1 complete -> [3,5,8,9,10,11] running, [12] added
...

```

The batch stays full. No wasted slots. As soon as a request finishes, another takes its place.

Throughput improvement:

- Static: 56% utilization (our earlier example)
- Continuous: 95%+ utilization (limited only by scheduling overhead)

The Orca paper reported 36.9x throughput improvement over FasterTransformer on GPT-3 175B. The improvement is larger for:

- Variable output lengths (more early completions)
- High request rates (more waiting requests to fill slots)
- Larger batch sizes (more opportunity for replacement)

The mechanics require some care:

- **KV cache management:** Each sequence needs its own KV cache. When a sequence finishes, its cache must be freed for reuse.
- **Attention masking:** Sequences in the same batch may have different lengths. The attention mask must prevent cross-sequence attention.
- **Position encoding:** Each sequence tracks its own position, not a global batch position.

These complexities are why continuous batching wasn't the default from the start. The implementation is harder than static batching. But the throughput gains make it essential for production.

Scheduler Design

The scheduler decides which requests run and in what order. Different policies optimize for different objectives: throughput, latency, fairness.

In this section we cover:

- Request states and lifecycle
- Scheduling policies
- Preemption and priorities

A request moves through states:

```
WAITING -> PREFILLING -> DECODING -> COMPLETED
      ↓
      PREEMPTED -> WAITING (re-queued)
```

WAITING: Request received, not yet started. Prompt tokens known, no KV cache allocated.

PREFILLING: Processing the prompt. Building the initial KV cache.

DECODING: Generating tokens one at a time. KV cache growing with each token.

COMPLETED: Hit stop token or max length. Ready for response.

PREEMPTED: Temporarily stopped to make room for higher-priority requests. KV cache may be kept or swapped.

The simplest policy is FCFS (First Come First Served):

```
class FCFSScheduler:
    def __init__(self, max_batch_size, max_tokens):
        self.waiting, self.running = deque(), []
        self.max_batch_size, self.max_tokens = max_batch_size, max_tokens
    def get_batch(self):
        while len(self.running) < self.max_batch_size and self.waiting:
            request = self.waiting.popleft()
            if sum(r.num_tokens for r in self.running) + request.num_tokens <= self.max_tokens:
                self.running.append(request)
        return self.running or None
```

FCFS is fair but not optimal. A long request can block many short ones.

Shortest Job First (SJF) minimizes average latency:

```
class SJFScheduler(FCFSScheduler):
    def add_request(self, request):
        self.waiting.append(request)
        self.waiting.sort(key=lambda r: r.estimated_tokens)
```

But we don't know output lengths in advance. Heuristics based on prompt similarity or request history can estimate.

Priority scheduling for latency-sensitive requests:


```

class PriorityScheduler:
    def add_request(self, request, priority=0):
        heappush(self.waiting, (-priority, request.arrival_time, request))

    def get_batch(self):
        batch = []
        while len(batch) < self.max_batch_size and self.waiting:
            batch.append(heapop(self.waiting)[2])
        return batch

```

Production systems use hybrid policies:

- Priority levels (premium vs free tier)
- Deadline-aware scheduling (SLA guarantees)
- Fairness constraints (prevent starvation)

Preemption handles overload. When a high-priority request arrives and the batch is full, a lower-priority request can be preempted:

```

def preempt_if_needed(self, new_request):
    if not self.can_fit(new_request):
        victim = min(self.running, key=lambda r: r.priority)
        if victim.priority < new_request.priority:
            self.swap_out(victim)
            self.waiting.appendleft(victim)
            self.running.remove(victim)

```

Swapping saves the KV cache to CPU memory or disk. The preempted request can resume later without re-processing its prompt.

Radix Cache

Many requests share common prefixes: system prompts, few-shot examples, shared documents. Radix caching reuses KV cache across requests that share prefixes, dramatically reducing redundant computation.

In this section we cover:

- Prefix sharing opportunity
- Radix tree data structure for KV cache
- Matching and eviction policies

Consider a chatbot with a system prompt:

System: You are a helpful assistant. Be concise and accurate.

User: What is the capital of France?

User: What is the capital of Germany?

User: What is the capital of Spain?

All three requests share the system prompt prefix. Computing the KV cache for “You are a helpful assistant. Be concise and accurate.” three times is wasteful.

With radix caching: 1. First request: compute KV for system prompt + question 1 2. Second request: reuse system prompt KV, compute only question 2 3. Third request: reuse system prompt KV, compute only question 3

Savings: 2x the system prompt compute eliminated.

The radix tree stores prefixes:

Each node stores children (next tokens), the KV cache at that prefix, and a reference count:

```
class RadixNode:
    def __init__(self):
        self.children = {}
        self.kv_cache = None
        self.ref_count = 0
```

The match_prefix method walks the tree to find the longest cached prefix:

```
def match_prefix(self, tokens):
    node, matched = self.root, 0
    for token in tokens:
        if token in node.children:
            node = node.children[token]
            matched += 1
        else:
            break
    return node, matched
```

When a new request arrives: 1. Match its tokens against the radix tree 2. If prefix found, reuse the cached KV 3. Compute KV only for unmatched tokens 4. Insert the full sequence into the tree

```
def prefill_with_cache(request):
    node, matched = cache.match_prefix(request.tokens)
    request.kv_cache = node.kv_cache[:matched] if matched > 0 else None
    request.position = matched
    new_kv = compute_kv(request.tokens[matched:], request.position)
    request.kv_cache = concat(request.kv_cache, new_kv)
    cache.insert(request.tokens, request.kv_cache)
```

Eviction is necessary when memory fills. LRU (Least Recently Used) works well:

```
def evict_if_needed(self, required_memory):
    while self.used_memory + required_memory > self.max_memory:
        victim = self.lru_queue.pop()
        if victim.ref_count == 0:
            self.remove_node(victim)
            self.used_memory -= victim.kv_cache.memory
```

Only nodes with zero reference count can be evicted. Active requests increment ref_count; completed requests decrement it.

Real-world savings depend on workload:

- Chat with system prompts: 20-40% compute saved
- Multi-turn conversations: 50%+ saved (each turn reuses previous)
- Batch processing with templates: 70%+ saved

SGLang introduced RadixAttention, a production implementation of these ideas. It's now the standard approach for prefix caching.

Memory Management

KV cache memory management is the hardest part of continuous batching. Sequences have variable lengths. Memory fragments. Allocation failures cause request stalls.

In this section we cover:

- The memory fragmentation problem
- PagedAttention and block-based allocation
- Copy-on-write for efficient branching

Naive allocation assigns contiguous memory for each sequence's KV cache:

```
def allocate_kv_cache(request, max_seq_len):
    cache_size = max_seq_len * num_layers * num_heads * head_dim * 2
    return torch.empty(cache_size, device='cuda')
```

Problem: we don't know the final sequence length. If we allocate for `max_seq_len`, we waste memory on short sequences. If we allocate just enough, we need to reallocate as sequences grow.

With variable-length sequences coming and going:

Memory: [####A####] [####B####] [____free____] [####C####] [free] [####D####]

Fragmentation leaves holes that can't fit new sequences even though total free memory is sufficient.

PagedAttention (from vLLM) solves this with block-based allocation:

```
class PagedKVCache:
    def __init__(self, num_blocks, block_size, num_layers, num_heads, head_dim):
        self.block_size = block_size
        self.cache = torch.zeros(num_blocks, 2, num_layers, block_size, num_heads, head_dim)
        self.free_blocks, self.block_tables = list(range(num_blocks)), {}
    def allocate_block(self): return self.free_blocks.pop()
    def free_block(self, block_id): self.free_blocks.append(block_id)
```

Each sequence gets a block table mapping logical positions to physical blocks:

```
def allocate_for_sequence(self, seq_id, num_tokens):
    num_blocks_needed = (num_tokens + self.block_size - 1) // self.block_size
    blocks = [self.allocate_block() for _ in range(num_blocks_needed)]
    self.block_tables[seq_id] = blocks
```

Memory layout with paging:

Physical blocks: [Block0] [Block1] [Block2] [Block3] [Block4] [Block5]

Sequence A: [0, 3] -> Blocks 0, 3

Sequence B: [1, 4] -> Blocks 1, 4

Sequence C: [2, 5] -> Blocks 2, 5

No fragmentation. Any free block can be used by any sequence. Memory waste is at most one block per sequence (internal fragmentation in the last block).

Attention with paged cache requires custom kernels that handle the indirection:

```
def paged_attention(q, block_table, kv_cache, seq_len):
    output = torch.zeros_like(q)
    for block_idx, block_id in enumerate(block_table):
        k_block, v_block = kv_cache[block_id, 0], kv_cache[block_id, 1]
        start, end = block_idx * block_size, min((block_idx + 1) * block_size, seq_len)
        output += attention_block(q, k_block, v_block, start, end)
    return output
```

FlashInfer handles this efficiently with its paged attention kernels.

Copy-on-write enables efficient branching for beam search and parallel sampling:

```
def fork_sequence(self, src_seq_id, dst_seq_id):
    self.block_tables[dst_seq_id] = self.block_tables[src_seq_id].copy()
    for block_id in self.block_tables[dst_seq_id]: self.ref_counts[block_id] += 1

def extend_sequence(self, seq_id):
    last = self.block_tables[seq_id][-1]
    if self.ref_counts[last] > 1:
        new = self.allocate_block(); self.cache[new] = self.cache[last].clone()
        self.block_tables[seq_id][-1], self.ref_counts[last] = new, self.ref_counts[last] - 1
```

Multiple sequences can share blocks until one needs to modify. Only then is the block copied.

Chapter 8: Advanced Scheduling

Continuous batching is the foundation. This chapter covers the techniques that make it work well in practice.

Long prompts are problematic for continuous batching: a 32K token prefill blocks decode tokens for other requests. Chunked prefill breaks long prefills into pieces that interleave with decodes.

Mixing prefill and decode in the same batch is tricky because they have different computational characteristics. We'll see how to handle this efficiently.

CUDA Graphs eliminate kernel launch overhead, which matters for small batch decode where launch time can exceed compute time.

Overlap scheduling hides the CPU scheduling overhead behind GPU computation, keeping the pipeline full.

These techniques are what separate a basic continuous batching implementation from a production-ready system.

Chunked Prefill

A 32K token prompt takes significant time to prefill. During that time, no decode tokens can be generated. Other users wait. Chunked prefill fixes this by processing long prompts in pieces.

In this section we cover:

- Why long prefills hurt latency
- Breaking prefill into chunks
- Scheduling chunks with decode tokens

The problem: prefill is compute-bound and takes time proportional to prompt length squared (due to attention). For a 32K prompt on an H100:

Prefill time $\sim (32K)^2 * 80 \text{ layers} * \text{ops} / \text{FLOPS} \sim 2\text{-}5 \text{ seconds}$

During those seconds, decode requests pile up. Users generating tokens see their latency spike.

The solution: don't prefill the whole prompt at once. Break it into chunks:

```
class ChunkedPrefillScheduler:
    def __init__(self, chunk_size=512):
        self.chunk_size = chunk_size
```

```
def prepare_request(self, request):
    request.chunks = [request.prompt_tokens[i:i+self.chunk_size]
                      for i in range(0, len(request.prompt_tokens), self.chunk_size)]
    request.current_chunk = 0
    request.state = 'chunked_prefill' if len(request.chunks) > 1 else 'prefill'
```

Each iteration processes one chunk of prefill tokens:

```
def step(self, request):
    if request.state == 'chunked_prefill':
        chunk = request.chunks[request.current_chunk]
        request.kv_cache = concat(request.kv_cache, compute_kv(chunk, request.position))
        request.position += len(chunk)
        request.current_chunk += 1
        if request.current_chunk >= len(request.chunks):
            request.state = 'decode'
    elif request.state == 'decode':
        request.output_tokens.append(decode_step(request))
```

Now a 32K prompt with 512-token chunks takes 64 iterations instead of 1. Between each chunk, decode tokens can run.

Latency distribution changes:

Without chunking:

- Prefill request: 3000 ms
- Decode requests: blocked, then burst

With chunking (512 tokens):

- Prefill request: 3000 ms total, 47 ms per chunk
- Decode requests: 47 ms delay max per iteration

The P99 latency for decode tokens improves dramatically.

Chunk size is a tradeoff:

- Smaller chunks: lower decode latency, more scheduling overhead
- Larger chunks: higher decode latency, better compute efficiency

Typical production values: 256-1024 tokens per chunk.

There's a subtlety with attention. Each chunk must attend to all previous chunks. The KV cache from previous chunks must be available. This works naturally with our incremental KV cache design - each chunk extends the existing cache.

Mixed Batches

With chunked prefill, a single batch might contain both prefill tokens and decode tokens. These have very different computational characteristics. Handling them efficiently requires care.

In this section we cover:

- The difference between prefill and decode computation
- Batching strategies for mixed workloads
- Attention with mixed sequence lengths

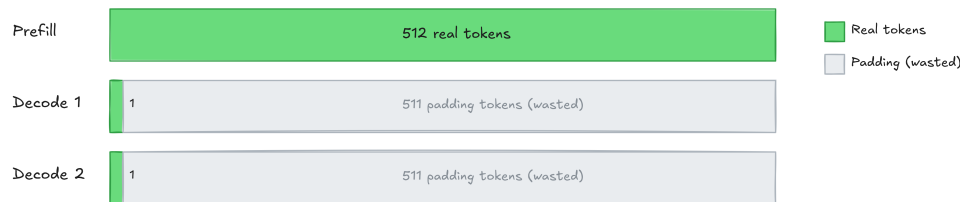
Recall the difference:

- **Prefill:** Many tokens per sequence. GEMM (matrix-matrix multiply). Compute-bound.
- **Decode:** One token per sequence. GEMV (matrix-vector multiply). Memory-bound.

If we naively batch them together, we pad decode sequences to match prefill length. Consider a prefill with 512 tokens and two decode requests with 1 token each. With padding, we process 512 tokens for prefill, but the decode sequences each get padded to 512 tokens despite only having 1 real token. Efficiency: $(512 + 1 + 1) / (512 * 3) = 33\%$. Two-thirds of compute wasted on padding.

Naive Batching: Padding Waste

Batch with 1 prefill (512 tokens) + 2 decode (1 token each)



Efficiency: $(512 + 1 + 1) / (512 \times 3) = 33\%$

Two-thirds of compute wasted on padding!

Figure 4: Naive batching wastes compute on padding

Better approach: batch them separately in the same forward pass:

```
def mixed_forward(prefill_batch, decode_batch, model):
    all_inputs = concat([prefill_batch.input_ids, decode_batch.input_ids])
    hidden = model.embed(all_inputs)
    for layer in model.layers:
        prefill_out = layer.attention(hidden[:prefill_batch.num_tokens], prefill_batch.kv_cache)
        decode_out = layer.attention(hidden[prefill_batch.num_tokens:], decode_batch.kv_cache)
        hidden = layer.ffn(concat([prefill_out, decode_out]))
    return model.lm_head(hidden)
```

The attention is computed separately (different KV caches, different sequence lengths), but the FFN operates on the concatenated tokens.

This works because:

- Embedding is per-token, no batch dimension issues
- FFN is per-token, no batch dimension issues
- Attention must be separate anyway (different KV caches)

FlashInfer provides optimized kernels for this pattern:

```
prefill_wrapper = flashinfer.BatchPrefillWithPagedKVCacheWrapper(workspace)
decode_wrapper = flashinfer.BatchDecodeWithPagedKVCacheWrapper(workspace)

def attention_layer(q_prefill, q_decode, kv_cache):
    out_prefill = prefill_wrapper.forward(q_prefill, kv_cache) if q_prefill else None
    out_decode = decode_wrapper.forward(q_decode, kv_cache) if q_decode else None
    return concat([out_prefill, out_decode])
```

Scheduling the mix is another decision:

```
def schedule_mixed_batch(self):
    budget = self.max_tokens_per_step
    prefill_tokens = []
    for req in self.prefill_queue:
        chunk = req.next_chunk(max_size=budget)
        if chunk: prefill_tokens.append(chunk); budget -= len(chunk)
    decode_requests = [r for r in self.decode_queue[:budget]]
    return MixedBatch(prefill_tokens, decode_requests)
```

The budget ensures total tokens don't exceed memory limits. Prefill chunks can be sized to leave room for decode tokens.

CUDA Graphs

Kernel launch overhead is 2-10 microseconds per launch. A decode step might have 300+ kernel launches. For small batches, launch overhead exceeds compute time. CUDA Graphs fix this.

In this section we cover:

- The kernel launch problem
- Recording and replaying CUDA Graphs
- Limitations and workarounds

The problem in numbers:

Batch size 1, 7B model:

- Compute time: ~5 ms
- Launch overhead: 300 launches * 5 us = 1.5 ms
- Overhead: 30% of total time

Batch size 32:

- Compute time: ~20 ms

- Launch overhead: $300 * 5 \text{ us} = 1.5 \text{ ms}$
- Overhead: 7.5% of total time

Launch overhead is roughly constant, so it matters more for small batches.

CUDA Graphs record a sequence of operations, then replay the entire sequence with a single launch:

```
class CUDAGraphRunner:
    def __init__(self, model, input_shape):
        self.graph, self.static_input = torch.cuda.CUDAGraph(), torch.zeros(input_shape, device='cuda')
        with torch.cuda.graph(self.graph):
            self.static_output = model(self.static_input)

    def forward(self, input_tensor):
        self.static_input.copy_(input_tensor)
        self.graph.replay()
        return self.static_output.clone()
```

Recording captures all CUDA operations. Replay executes them without CPU involvement.

The speedup for small batches is dramatic:

Batch size 1:

- Without graph: 6.5 ms
- With graph: 5.1 ms
- Speedup: 1.27x

Batch size 4:

- Without graph: 8.2 ms
- With graph: 6.1 ms
- Speedup: 1.34x

Limitations make graphs tricky:

Fixed shapes: Graphs capture specific tensor shapes. If batch size or sequence length changes, you need a different graph.

```
class MultiShapeGraphRunner:
    def __init__(self, model, shapes):
        self.graphs = {shape: self._capture_graph(model, shape) for shape in shapes}

    def forward(self, input_tensor):
        shape = tuple(input_tensor.shape)
        return self.graphs[shape].replay(input_tensor) if shape in self.graphs else self.model(input_tensor)
```

Production systems pre-capture graphs for common shapes: batch sizes 1, 2, 4, 8, 16, 32.

No dynamic control flow: If your model has data-dependent branches, graphs can't capture it.

Memory constraints: Each graph allocates its own intermediate buffers. Many graphs consume significant memory.

Warmup cost: Capturing a graph requires running the model once. First request to a new shape pays this cost.

For decode steps, shapes are predictable: `batch_size` varies, but sequence length per token is always 1. This makes graphs practical.

For prefill, shapes vary more (different prompt lengths). Graphs help less here, and chunked prefill makes shapes more predictable.

Overlap Scheduling

The CPU scheduler needs time to decide what runs next. If scheduling happens after GPU work finishes, the GPU sits idle. Overlap scheduling runs scheduling in parallel with GPU computation.

In this section we cover:

- CPU scheduling overhead
- Pipelining CPU and GPU work
- Implementation patterns

The sequential approach alternates between GPU compute and CPU scheduling. GPU compute runs, then CPU scheduling, then GPU compute again. If scheduling takes 2 ms and compute takes 10 ms, we lose $2/12 = 17\%$ of potential throughput.

The overlapped approach runs CPU scheduling in parallel with GPU compute. While step N runs on GPU, the CPU prepares step N+1. While step N+1 runs, the CPU prepares step N+2. The GPU never waits.

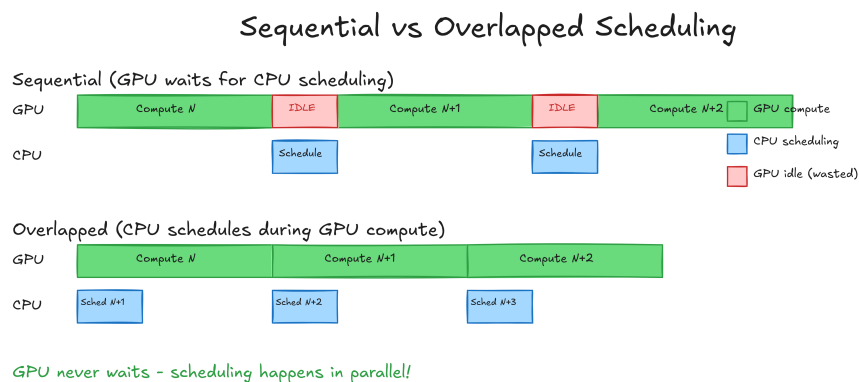


Figure 5: Sequential vs overlapped scheduling

Implementation requires two threads (or async execution):

```
class OverlapScheduler:
    def __init__(self, model):
        self.model, self.next_batch, self.lock = model, None, threading.Lock()
```

The run loop overlaps GPU compute with CPU scheduling:

```
def run(self):
    self.next_batch = self.schedule()
    while True:
        with self.lock: batch = self.next_batch
        self.schedule_thread = threading.Thread(target=self._schedule_next)
        self.schedule_thread.start()
        if batch: self.process_output(self.model.forward(batch))
        self.schedule_thread.join()
```

The GPU work and CPU scheduling overlap. The `join()` at the end ensures scheduling finished before we use the result.

CUDA streams enable finer-grained overlap:

```
compute_stream, transfer_stream = torch.cuda.Stream(), torch.cuda.Stream()

def overlapped_forward(current_batch, next_batch):
    with torch.cuda.stream(compute_stream): output = model(current_batch)
    with torch.cuda.stream(transfer_stream): next_batch.to('cuda', non_blocking=True)
    compute_stream.synchronize()
    return output
```

Data transfer for the next batch happens while the current batch computes.

The scheduling work includes:

- Checking which sequences completed
- Selecting next sequences to run
- Preparing input tensors (token IDs, positions, attention masks)
- Updating KV cache pointers

For complex schedulers with radix matching or priority queues, this can take 1-5 ms.

Measurements show:

Configuration	Throughput
Sequential scheduling	1000 tok/s
Overlapped scheduling	1150 tok/s
Improvement	15%

The improvement is larger when:

- Scheduling is complex (radix cache, priorities)
- Batch sizes are small (less GPU work to hide overhead)

- Request rates are high (more scheduling decisions)

SGLang implements overlap scheduling by default. The scheduler runs in a separate thread, preparing the next batch while the current one executes.

Chapter 9: Mixture of Experts

Mixture of Experts (MoE) is an architectural approach that increases model capacity without proportionally increasing compute. Instead of one large FFN, MoE uses many smaller expert FFNs, with a router selecting which experts process each token.

This changes the inference characteristics. Prefill and decode still exist. Memory bandwidth still dominates decode. But the compute pattern is different: instead of dense matrix multiplications, we route tokens to sparse subsets of experts.

Understanding MoE is essential because many frontier models use this architecture. Mixtral, Qwen-MoE, DeepSeek, and others achieve better quality per FLOP through sparsity. The same inference principles apply - we just need to understand how routing affects batching and memory access patterns.

MoE Basics

A standard transformer uses one FFN per layer. Every token goes through the same parameters. Mixture of Experts replaces that single FFN with many smaller FFNs called experts, plus a router that decides which experts each token uses.

In this section we cover:

- The MoE architecture
- Router mechanics
- Sparse vs dense computation

The standard FFN:

```
def ffn(x, w1, w2):  
    return w2 @ gelu(w1 @ x)
```

Every token uses w1 and w2. For a 7B model, these weights might be 4096 x 11008 and 11008 x 4096.

The MoE version:

```
class MoELayer:  
    def __init__(self, d_model, d_ff, num_experts, top_k):  
        self.router = nn.Linear(d_model, num_experts)  
        self.experts = nn.ModuleList([FFN(d_model, d_ff) for _ in range(num_experts)])  
        self.top_k = top_k
```

The forward pass routes tokens to their top-k experts:

```
def forward(self, x):
    router_logits = self.router(x)
    weights, indices = torch.topk(router_logits, self.top_k)
    weights = F.softmax(weights, dim=-1)
    output = torch.zeros_like(x)
    for i, expert in enumerate(self.experts):
        mask = (indices == i).any(dim=-1)
        if mask.any(): output[mask] += self._weighted_expert(expert, x[mask], weights[mask], i)
    return output
```

Key parameters:

- `num_experts`: How many expert FFNs (typically 8-64)
- `top_k`: How many experts each token uses (typically 1-2)
- `d_ff`: Hidden dimension per expert (smaller than dense FFN)

Qwen3-30B-A3B has 8 experts with `top_k=2`. Each expert contains a portion of the FFN parameters. Total parameters: 30B. But each token only uses 2 experts, activating roughly 3B parameters per token - hence the “A3B” (Active 3B) in the name.

The router produces logits for each expert:

```
router_logits = self.router(x)
```

This is a simple linear projection from `d_model` to `num_experts`. For Qwen3-30B-A3B: 4096 -> 8.

Top-k selection picks which experts to use:

```
weights, indices = torch.topk(router_logits, self.top_k)
```

If `top_k=2` and we have 8 experts, each token picks its 2 highest-scoring experts.

The weights are normalized:

```
weights = F.softmax(weights, dim=-1)
```

These become mixing coefficients. If expert 3 got score 0.7 and expert 5 got score 0.3, the output is $0.7 * \text{expert_3}(x) + 0.3 * \text{expert_5}(x)$.

Why MoE helps:

Dense 70B model:

- Parameters: 70B
- Active per token: 70B
- Memory: 140 GB (fp16)

MoE 30B-A3B model (Qwen3):

- Parameters: 30B total
- Active per token: ~3B (2 experts)
- Memory: 60 GB (fp16)

MoE gives you more total capacity with less active compute per token. The tradeoff is complexity in routing and expert load balancing.

The sparse computation pattern creates challenges. In dense models, every token does the same computation - perfect for batched matrix multiplies. In MoE, different tokens go to different experts:

```
Token 1 -> Expert 2, Expert 5
Token 2 -> Expert 1, Expert 5
Token 3 -> Expert 2, Expert 7
Token 4 -> Expert 3, Expert 6
```

Naive implementation loops over experts. This is slow because:

1. Small batches per expert (few tokens per expert)
2. Many kernel launches
3. Poor GPU utilization

The solution is grouped GEMM or expert-parallel kernels that we cover in the next section.

MoE Inference

MoE models present unique inference challenges. Expert parameters must be loaded when needed, tokens must be routed efficiently, and load imbalance can waste compute. This section covers practical techniques for MoE inference.

In this section we cover:

- Expert caching strategies
- Efficient token routing
- Load balancing considerations

The memory problem is severe. Qwen3-30B-A3B has 30B parameters. At fp16, that is 60 GB - consuming most of a single 80GB H100. With multiple concurrent requests, fitting all experts plus KV caches in VRAM gets tight. The B200's 192 GB provides more headroom.

Expert caching treats GPU memory like a cache:

```
class ExpertCache:
    def __init__(self, cache_size, expert_loader):
        self.cache_size, self.cached_experts, self.lru_order, self.expert_loader = cache_size, [], []
    def get_expert(self, expert_id):
        if expert_id in self.cached_experts:
            self.lru_order.remove(expert_id); self.lru_order.append(expert_id); return self.cached_experts[expert_id]
        if len(self.cached_experts) >= self.cache_size: del self.cached_experts[self.lru_order[-1]]
        self.cached_experts[expert_id] = self.expert_loader(expert_id); self.lru_order.append(expert_id)
        return self.cached_experts[expert_id]
```

With 8 experts and cache size 4, we keep the 4 most recently used experts on GPU. When a token needs an uncached expert, we evict the least recently used one and load the new expert from CPU memory or disk.

This works because expert usage is often skewed. Some experts are popular, others rarely used. A well-trained router tends to specialize experts, creating predictable access patterns.

Prefetching improves latency:

```
class PrefetchingExpertCache:
    def __init__(self, cache, predictor):
        self.cache, self.predictor, self.prefetch_stream = cache, predictor, torch.cuda.Stream()
    def forward(self, x, router_logits):
        weights, indices = torch.topk(router_logits, k=2)
        with torch.cuda.stream(self.prefetch_stream):
            for eid in self.predictor(router_logits):
                if eid not in self.cache.cached_experts: self.cache.prefetch(eid)
        loaded = [self.cache.get_expert(i) for i in indices.unique().tolist()]
        return self.compute(x, weights, indices, loaded)
```

While computing with current experts, we predict and prefetch likely next experts on a separate CUDA stream.

Efficient token routing avoids the naive loop-over-experts pattern:

```
def grouped_expert_forward(x, router_logits, experts, top_k):
    batch_size, d_model = x.shape
    weights, indices = torch.topk(router_logits, top_k)
    weights = F.softmax(weights, dim=-1)
    sorted_indices, sort_order = indices.flatten().sort()
    token_indices = torch.arange(batch_size).repeat_interleave(top_k)[sort_order]
    expert_boundaries = torch.searchsorted(sorted_indices, torch.arange(len(experts) + 1, device=x.device))
    x_sorted = x[token_indices]
```

Each expert processes its contiguous chunk of sorted tokens:

```
output_sorted = torch.zeros(batch_size * top_k, d_model, device=x.device)
for i, expert in enumerate(experts):
    start, end = expert_boundaries[i], expert_boundaries[i + 1]
    if start < end: output_sorted[start:end] = expert(x_sorted[start:end])
output_unsorted = output_sorted[sort_order.argsort()].view(batch_size, top_k, d_model)
return (output_unsorted * weights.unsqueeze(-1)).sum(dim=1)
```

The key insight: sort tokens by expert assignment, then process contiguous chunks. Each expert gets a single batched GEMM instead of scattered individual tokens.

For large batches, this grouping makes MoE nearly as efficient as dense models:

Batch 1:	0.25 tokens/expert avg	-> very small GEMMs, poor efficiency
Batch 64:	16 tokens/expert avg	-> decent GEMM sizes, reasonable efficiency
Batch 256:	64 tokens/expert avg	-> good GEMM sizes, high efficiency

Load balancing matters for throughput. If all tokens route to the same 2 experts:

Expert 0:	100% load
Expert 1:	100% load
Expert 2-7:	0% load

Six experts sit idle. Total compute is 2x what it should be because we cannot parallelize across experts.

Training includes auxiliary losses to encourage balanced expert usage. At inference, we live with whatever routing the trained model produces. Some techniques help:

```
def load_balanced_routing(router_logits, expert_capacity, num_experts):
    weights, indices = torch.topk(router_logits, k=2)
    expert_counts = torch.zeros(num_experts, device=router_logits.device)
    for i, token_experts in enumerate(indices):
        for k, eid in enumerate(token_experts):
            if expert_counts[eid] >= expert_capacity:
                eid = (expert_counts < expert_capacity).nonzero()[0].item()
                expert_counts[eid] += 1; indices[i, k] = eid
    return weights, indices
```

Expert capacity limits how many tokens each expert can handle. Overflow tokens get rerouted to less busy experts. This trades off quality (suboptimal routing) for efficiency (balanced load).

Chapter 10: Production Server

This chapter brings together everything we have built. The scheduler, memory manager, and inference engine combine into a working server that handles real HTTP requests.

We start with API design: what requests look like, how streaming works, and the contract between client and server.

FastAPI provides the HTTP layer. We will see how async Python handles concurrent requests without blocking, and how the server communicates with the inference engine.

Tokenization is surprisingly expensive. A separate tokenizer worker pool prevents tokenization from blocking GPU work.

Finally, we benchmark the complete system. Throughput, latency distributions, and scaling behavior tell us if we built something worth using.

By the end, you will have a complete mental model of how production LLM inference servers work, from HTTP request to generated token.

API Design

Before writing server code, we need to define what requests look like. The API determines what clients can request and what the server promises to deliver.

In this section we cover:

- Request and response schemas
- Streaming vs non-streaming responses
- Error handling and timeouts

Request Schema

A completion request needs just enough information to generate text: the prompt, how many tokens to generate, and sampling parameters. The minimal viable request looks like this:

```
@dataclass
class CompletionRequest:
    prompt: str
    max_tokens: int = 256
    temperature: float = 1.0
```

```
top_p: float = 1.0
stream: bool = False
```

Every field except `prompt` has a default. This means clients can send just `{"prompt": "Hello"}` and get a reasonable response. Temperature controls randomness: 0 is deterministic, higher values add variety. Top-p (nucleus sampling) truncates the probability distribution to the most likely tokens.

The `stream` flag determines whether the server waits until generation completes or sends tokens as they are produced. Streaming matters for user experience: seeing text appear word-by-word feels responsive even when total latency is the same.

Response Schema

The non-streaming response bundles everything together:

```
@dataclass
class CompletionResponse:
    text: str
    tokens_generated: int
    prompt_tokens: int
    finish_reason: str
```

The `finish_reason` tells clients why generation stopped: `"stop"` means a stop token was hit, `"length"` means `max_tokens` was reached. This matters for applications that need to detect truncation.

OpenAI Compatibility

The OpenAI API has become the de facto standard. Compatibility means existing tools, libraries, and scripts work without modification. The core structures are more verbose but follow the same pattern. See the full schemas in `ch10/api_design.py`.

The key additions for OpenAI compatibility are the `messages` array (for chat) instead of raw prompts, and the nested `choices` array in responses. Chat messages carry a role (system, user, assistant) that tells the model how to interpret each piece of context.

```
@dataclass
class OpenAICompletionRequest:
    model: str
    messages: list[dict]
    max_tokens: int | None = None
    temperature: float = 1.0
    stream: bool = False
```

Streaming with Server-Sent Events

Streaming uses Server-Sent Events (SSE), an HTTP standard for pushing data from server to client. Each chunk is a JSON object prefixed with `data::`:

```
data: {"choices": [{"delta": {"content": "Hello"}}]}
```

```
data: {"choices": [{"delta": {"content": " world"}}]}
```

```
data: {"choices": [{"delta": {}, "finish_reason": "stop"}]}
```

```
data: [DONE]
```

The `delta` field contains only what changed since the last chunk. For content tokens, that is the new text. The final chunk has an empty delta and a `finish_reason`. The `[DONE]` marker signals the stream ended cleanly.

Why this format? SSE works over standard HTTP, needs no special client libraries, and handles reconnection gracefully. WebSockets would be faster but require more infrastructure. For LLM inference where token generation dominates latency, SSE overhead is negligible.

The streaming implementation yields chunks as an async generator:

```
async def stream_response(request_id: str, engine: Engine):
    async for token in engine.generate_stream(request_id):
        chunk = {"choices": [{"delta": {"content": token}}]}
        yield f"data: {json.dumps(chunk)}\n\n"
    yield "data: [DONE]\n\n"
```

Error Handling

Clients expect specific HTTP status codes with structured error bodies. The error format includes a message, type, and code:

```
def create_error_response(message: str, code: int) -> dict:
    return {"error": {"message": message, "code": code}}
```

Common errors fall into categories. 400 errors mean the client sent something invalid: missing required fields, out-of-range parameters, or malformed JSON. 401 and 403 handle authentication. 429 signals rate limiting: the client is sending too many requests. 500 errors mean the server broke.

Rate limiting deserves special attention. Without it, a single client can monopolize GPU resources. The server tracks requests per client and returns 429 when limits are exceeded. The response includes a `Retry-After` header telling clients when to try again.

Timeouts

Long-running requests need timeout protection. Three timeouts matter:

`request_timeout`: Total time allowed (e.g., 300 seconds)

`first_token_timeout`: Time until first token (e.g., 60 seconds)

`inter_token_timeout`: Maximum gap between tokens (e.g., 30 seconds)

The `first_token_timeout` catches requests stuck in a queue. If the server is too busy to start generating within a minute, the request should fail rather than hang. `Inter_token_timeout` catches cases where generation stalls mid-stream. The overall `request_timeout` caps total duration regardless of progress.

Validation

Request validation catches bad inputs before they reach the inference engine:

```
def validate_request(request: CompletionRequest):
    if not request.prompt:
        raise APIError("Prompt cannot be empty", 400)
    if request.max_tokens < 1 or request.max_tokens > 4096:
        raise APIError("max_tokens out of range", 400)
```

Validation happens synchronously before any async work starts. Rejecting invalid requests immediately saves GPU cycles and gives clients fast feedback. The full validation logic checks temperature bounds, top_p ranges, and stop sequence formats. See `ch10/api_design.py` for the complete implementation.

FastAPI Server

FastAPI handles HTTP requests asynchronously. The server receives requests, validates them, passes them to the inference engine, and streams results back. Understanding the async flow is essential.

In this section we cover:

- Async request handling
- Communication with the inference engine
- Connection management

The Async Model

Python's `asyncio` runs many concurrent operations on a single thread. When one request waits for I/O or the inference engine, others can proceed. This is perfect for an inference server: most time is spent waiting for the GPU, not doing CPU work.

FastAPI builds on this. Each request handler is an async function. The framework schedules them automatically, interleaving execution at await points. A server can handle thousands of concurrent connections without spawning thousands of threads.

Server Architecture

The server needs two things: a way to submit requests to the inference engine and a way to receive tokens back. Async queues connect these pieces.

```
class Server:
    def __init__(self, engine: Engine):
        self.engine = engine
        self.request_queues: dict[str, asyncio.Queue] = {}
```

Each active request gets its own queue. When the engine generates a token, it pushes to that queue. When the HTTP handler wants to stream a response, it pulls from the queue. This decouples the inference thread from the async event loop.

Submitting a request creates the queue and tells the engine to start generating:

```
async def submit_request(self, request: CompletionRequest) -> str:
    request_id = str(uuid.uuid4())
    self.request_queues[request_id] = asyncio.Queue()
    await self.engine.add_request(request_id, request.prompt, ...)
    return request_id
```

Reading tokens is an async generator that yields until the queue receives a None sentinel:

```
async def get_tokens(self, request_id: str):
    queue = self.request_queues[request_id]
    while True:
        token = await queue.get()
        if token is None:
            break
        yield token
```

Engine Thread Communication

The inference engine runs in a separate thread because GPU operations block. Python's GIL means we cannot run inference and handle HTTP on the same thread efficiently. The engine thread runs a loop: fetch a batch from the scheduler, run forward pass, emit tokens.

The tricky part is crossing the thread boundary. The engine thread cannot directly await on asyncio objects. Instead, it uses `call_soon_threadsafe` to schedule queue operations on the event loop:

```
def emit_token(self, request_id: str, token: str):
    queue = self.output_queues[request_id]
    self.loop.call_soon_threadsafe(queue.put_nowait, token)
```

This is the bridge between sync and async worlds. The engine thread calls `emit_token` synchronously, but the queue operation executes on the event loop thread. The HTTP handler's `await queue.get()` then receives the token.

HTTP Endpoints

The FastAPI endpoint ties everything together. Non-streaming requests wait for all tokens, then return a single response. Streaming requests return a `StreamingResponse` that yields chunks:

```
@app.post("/v1/completions")
async def completions(request: CompletionRequest):
    request_id = await server.submit_request(request)
    if request.stream:
        return StreamingResponse(stream_tokens(server, request_id), media_type="text/event-stream")
    tokens = [t async for t in server.get_tokens(request_id)]
    return CompletionResponse(text="".join(tokens))
```

The streaming generator formats tokens as SSE chunks. Each `yield` sends data to the client immediately without buffering.

Handling Disconnects

Clients may disconnect mid-stream. The browser tab closes, the network drops, or the client gives up. Without disconnect detection, the server keeps generating tokens nobody will receive, wasting GPU cycles.

FastAPI provides `request.is_disconnected()` to check connection state:

```
async for token in server.get_tokens(request_id):
    if await raw_request.is_disconnected():
        server.cancel_request(request_id)
        return
    yield format_chunk(token)
```

When disconnection is detected, we cancel the request in the scheduler. This frees the KV cache blocks and removes the request from the batch. The engine stops generating tokens for this request on the next iteration.

Lifecycle Management

Startup and shutdown hooks initialize and clean up resources. The engine must be running before requests arrive, and it must drain gracefully on shutdown:

```
@app.on_event("startup")
async def startup():
    engine = Engine(model, scheduler)
    engine.set_event_loop(asyncio.get_event_loop())
    threading.Thread(target=engine_loop, daemon=True).start()
    app.state.server = Server(engine)
```

The daemon thread means the engine dies when the main process exits. For graceful shutdown, we stop accepting new requests and wait for active ones to complete before terminating the engine thread.

Single Worker

The server runs with one worker:

```
uvicorn server:app --host 0.0.0.0 --port 8000 --workers 1
```

One worker is typical because the GPU is the bottleneck. Multiple workers would fight over GPU access, causing contention without improving throughput. The async event loop handles concurrency; multiple processes do not help here.

See `ch10/fastapi_server.py` for the complete implementation including rate limiting, metrics collection, and error handling middleware.

Tokenizer Workers

Tokenization converts text to token IDs and back. This sounds simple, but it is CPU-bound work that can bottleneck throughput. Moving tokenization to a separate worker pool keeps it off the critical path.

In this section we cover:

- Why tokenization is expensive
- Worker pool architecture
- Async integration with the server

The Tokenization Bottleneck

Tokenization runs on the CPU. For a short prompt like “Hello, world!” it takes about 50 microseconds. For a 4K token prompt, it takes 2-5 milliseconds. These numbers seem small, but they add up.

Consider a busy server handling 100 requests per second with 1K token prompts. Tokenization alone takes roughly 100 milliseconds of CPU time per second. If this runs in the main async loop, it blocks other requests from making progress. The GPU sits idle waiting for tokens.

The fix is simple: offload tokenization to worker processes. The main loop submits text, workers encode it, and results come back asynchronously. The GPU never waits.

Process Pool vs Thread Pool

Python’s GIL makes threads ineffective for CPU-bound work. Multiple threads cannot execute Python bytecode simultaneously. Processes bypass this limitation: each has its own interpreter and GIL.

```
from concurrent.futures import ProcessPoolExecutor

class TokenizerPool:
    def __init__(self, tokenizer_path: str, num_workers: int = 4):
        self.executor = ProcessPoolExecutor(
            max_workers=num_workers,
            initializer=self._init_worker,
            initargs=(tokenizer_path,)
        )
```

The initializer function loads the tokenizer once per worker at startup. This avoids reloading for every request. Workers stay warm and ready.

Async Integration

The pool exposes async methods that wrap the synchronous executor:

```
async def encode(self, text: str) -> list[int]:
    loop = asyncio.get_event_loop()
    return await loop.run_in_executor(
        self.executor, self._encode, text
    )
```

`run_in_executor` submits work to the process pool and returns an awaitable. The event loop can handle other requests while tokenization runs. When the worker finishes, the awaitable resolves with the token IDs.

Batched Operations

For batch processing, we can tokenize multiple texts in parallel:

```
async def encode_batch(self, texts: list[str]) -> list[list[int]]:
    loop = asyncio.get_event_loop()
    futures = [
        loop.run_in_executor(self.executor, self._encode, t)
        for t in texts
    ]
    return await asyncio.gather(*futures)
```

This submits all texts simultaneously. Workers pick them up as they become available. The `gather` call waits for all results. For a 4-worker pool processing 4 texts, all complete in roughly the time of the slowest one, not 4x serial time.

Decoding Challenges

Decoding (tokens to text) happens during streaming. Each generated token needs to become a string chunk. Sending every token to the worker pool adds latency: a round-trip to a worker process takes time.

Two strategies help. First, batch multiple tokens before decoding. Wait for 8 tokens, then decode them together. This reduces round-trips at the cost of slightly delayed streaming.

```
async def get_tokens_batched(self, request_id: str, batch_size: int = 8):
    batch = []
    async for token_id in self.get_raw_tokens(request_id):
        batch.append(token_id)
        if len(batch) >= batch_size:
            text = await self.tokenizer_pool.decode(batch)
            yield text
            batch = []
```

Second, some tokenizers support incremental decoding. They track state across calls and handle partial tokens correctly. This avoids batching entirely but requires tokenizer support.

Worker Pool Sizing

Too few workers and tokenization becomes the bottleneck. Too many and you waste memory and suffer context switch overhead.

Rule of thumb: 2-4 workers per CPU core available

Typical configuration: 4-8 workers for a server

Memory per worker: 100-500 MB (tokenizer model)

The right number depends on prompt length distribution. Short prompts need fewer workers because tokenization is fast. Long prompts need more. If you see the token queue backing up while GPU utilization is low, add workers.

Integration with the Server

The server submits prompts for tokenization before passing them to the scheduler:

```
async def submit_request(self, request: CompletionRequest) -> str:
    token_ids = await self.tokenizer_pool.encode(request.prompt)
    request_id = str(uuid.uuid4())
    await self.engine.add_request(request_id, token_ids, ...)
    return request_id
```

Now the engine receives token IDs directly. No string processing happens on the inference thread. The tokenizer pool handles encoding asynchronously, the engine handles generation synchronously, and neither blocks the other.

See `ch10/tokenizer_workers.py` for the complete worker pool implementation including request tracking, statistics collection, and graceful shutdown.

Putting It Together

We have all the pieces: model, scheduler, memory manager, tokenizer pool, and HTTP server. This section shows how they connect into a complete system.

In this section we cover:

- System architecture
- Startup sequence
- Request flow from HTTP to token

System Architecture

The system has six components arranged in a pipeline. HTTP requests arrive at the FastAPI server. The server routes to two subsystems: the Tokenizer Pool (for encoding/decoding text) and the Request Manager (for tracking request state). The Request Manager feeds into the Scheduler, which decides what to run next. The Scheduler drives the Engine, which runs forward passes on the GPU. The Engine interacts with the KV Cache Manager to store and retrieve cached key-value pairs.

Each component has a single responsibility. The tokenizer pool handles text-to-tokens conversion. The scheduler handles batching decisions. The engine handles GPU execution. This separation makes the system testable and debuggable. When throughput drops, you can trace which component is the bottleneck.

The LLMServer Class

The server class orchestrates initialization. It creates each component in dependency order: model first (needed by everything), then KV cache (sized based on model dimensions), then scheduler (needs cache access), then tokenizer pool and request manager.

```
from ch10.model_loader import load_model
```

LLM Server Architecture

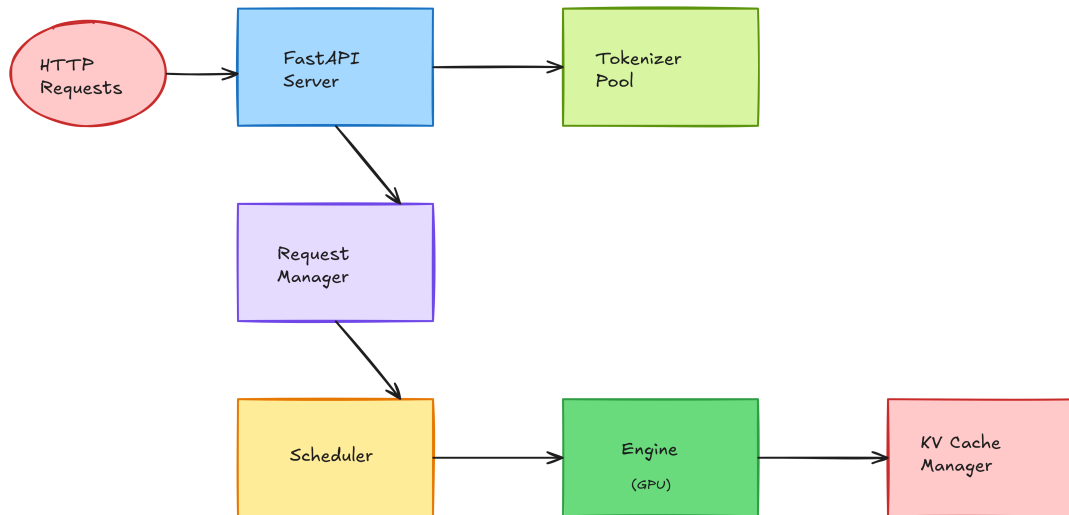


Figure 6: LLM server architecture showing component connections

```

class LLMServer:
    def __init__(self, model_path: str, config: ServerConfig):
        self.model_path = model_path
        self.config = config
        self.model = load_model(
            model_path,
            config.dtype,
            vocab_size=config.vocab_size,
        )

        engine_config = EngineConfig(
            max_batch_size=config.max_batch_size,
            max_seq_len=config.max_seq_len,
            max_total_tokens=config.max_tokens_per_batch,
            vocab_size=config.vocab_size,
            hidden_dim=config.hidden_dim,
            block_size=config.block_size,
            num_layers=config.num_layers,
            num_heads=config.num_heads,
            kv_cache_dtype=config.dtype,
            kv_cache_device=config.kv_cache_device,
        )
        self.engine = InferenceEngine(
            engine_config,

```

```

        model_fn=self.model,
    )
    self.tokenizer_pool = TokenizerPool(...)
    self.server = InferenceServer(...)

```

In the companion repo, `load_model` is defined in `ch10/model_loader.py`. It returns the callable demo model used by the chapter 10 server, so the example stays runnable without bundling a full pretrained checkpoint loader into the book repo.

The model loads first because its configuration (number of layers, head dimensions, vocabulary size) determines the size of everything else. The engine configuration, KV cache, and scheduler all depend on those values.

The Engine Loop

The engine runs in a dedicated thread, spinning a tight loop that fetches batches and runs forward passes:

```

def _engine_loop(self):
    while self.running:
        batch = self.scheduler.get_batch()
        if batch is None:
            time.sleep(0.001)
            continue
        outputs = self.model.forward(batch.input_ids, ...)
        for i, request_id in enumerate(batch.request_ids):
            self._process_output(request_id, outputs.logits[i])

```

The 1ms sleep when idle prevents busy-waiting. The batch object contains everything needed for one forward pass: input token IDs, position indices, block tables for paged attention, and sampling parameters per request.

After each forward pass, the engine samples the next token for each request and emits it to the corresponding output queue. If a request hits a stop condition (stop token or max length), it gets removed from the scheduler.

Startup Sequence

Server startup happens in phases. In a production system, model loading is the slow step: reading weights from disk and transferring them to GPU memory. In the companion repo, the `load_model` helper is a lightweight stand-in so the server wiring is runnable without shipping a full checkpoint.

```

async def create_server(model_path: str, config: ServerConfig):
    app = FastAPI()
    server = LLMServer(model_path, config)
    server.start()
    app.state.server = server
    app.state.loop = asyncio.get_event_loop()
    return app

```

Second, the engine thread starts. Third, the FastAPI event loop takes over, accepting HTTP connections. The `server.start()` call spawns the engine thread as a daemon, so it terminates when the main process exits.

Request Flow

A request flows through seven stages from HTTP to generated text:

1. HTTP request arrives at FastAPI
2. Tokenizer pool encodes prompt to token IDs (async)
3. Request manager creates tracking structures
4. Scheduler adds request to pending queue
5. Engine loop picks up request in next batch
6. Model runs forward pass, emits tokens
7. Tokens stream back via async queue to HTTP response

For streaming requests, stages 6-7 repeat for each generated token. For non-streaming, tokens accumulate and decode at the end.

The async boundaries matter. Stage 2 (tokenization) is async because it runs in a process pool. Stage 4 (scheduler add) is synchronous but fast. Stage 6-7 is where the engine thread and async loop communicate via `call_soon_threadsafe`.

Configuration

Configuration bundles all tunable parameters:

```
@dataclass
class ServerConfig:
    dtype: torch.dtype = torch.float16
    block_size: int = 16
    max_batch_size: int = 256
    max_tokens_per_batch: int = 4096
    tokenizer_workers: int = 4
```

Block size affects memory fragmentation (smaller blocks = less waste, more overhead). Max batch size limits concurrent requests. Max tokens per batch limits total computation per step. Tokenizer workers scale with CPU-bound tokenization load.

Running in Production

The server runs under uvicorn with a single worker:

```
uvicorn server:app --host 0.0.0.0 --port 8000 --workers 1
```

Single worker because the GPU is the bottleneck. The async event loop handles thousands of concurrent connections efficiently. Multiple workers would contend for GPU access without improving throughput.

For production deployments, add a reverse proxy (nginx), load balancer for multiple GPUs, and monitoring (Prometheus metrics). The core serving loop stays the same; infrastructure wraps around it.

See `ch10/integration.py` for the complete server implementation and `ch10/model_loader.py` for the demo `load_model` helper used by the book examples.

Benchmarking

A server is only as good as its performance under load. Benchmarking measures throughput, latency, and scaling behavior to verify the system works as expected.

In this section we cover:

- Key metrics and how to measure them
- Load testing methodology
- Interpreting results

Metrics That Matter

Four metrics characterize inference server performance:

Throughput: tokens/second (total across all requests)

Time to First Token (TTFT): how long users wait to see output start

Inter-Token Latency (ITL): gap between consecutive tokens during streaming

Utilization: GPU compute and memory usage

Throughput measures raw capacity. For batch inference workloads, this is the primary metric. A 7B model on H100 should achieve 4000-6000 tokens per second under load.

TTFT measures responsiveness. Users care about when text starts appearing, not just when it finishes. A 30ms TTFT feels instant; 500ms feels sluggish. TTFT depends on queue depth and prefill time.

ITL matters for streaming. Consistent 10ms ITL produces smooth-looking output. Sporadic 100ms gaps feel janky. ITL spikes often indicate scheduling problems or GC pauses.

Measuring Latency

A benchmark client needs to track timing at multiple points:

```
class BenchmarkClient:
    async def run_request(self, prompt: str, max_tokens: int):
        start, first_token_time, token_times = time.perf_counter(), None, []
        async for chunk in self.stream_request(prompt, max_tokens):
            now = time.perf_counter()
            if first_token_time is None: first_token_time = now
            token_times.append(now)
        return self.compute_metrics(start, first_token_time, token_times)
```

The first token time minus start gives TTFT. Consecutive token time differences give ITL samples. Total tokens divided by wall time gives throughput.

Load Generation

Benchmarking requires controlled load. A semaphore limits concurrent requests:

```
async def run_benchmark(client, prompts, duration_s):
    semaphore = asyncio.Semaphore(client.num_clients)
    async def bounded_request(prompt):
        async with semaphore: return await client.run_request(prompt, max_tokens=256)
    tasks = [bounded_request(p) for p in prompts]
    return aggregate_results(await asyncio.gather(*tasks))
```

The semaphore ensures exactly `num_clients` requests run simultaneously. Without it, all requests would launch immediately, overwhelming the server and producing unrealistic results.

Aggregating Results

Results need statistical aggregation. Means hide outliers; percentiles reveal them:

```
def aggregate_results(results):
    ttfts = [r.ttft_ms for r in results]
    return BenchmarkResult(
        mean_ttft_ms=statistics.mean(ttfts),
        p50_ttft_ms=statistics.median(ttfts),
        p99_ttft_ms=sorted(ttfts)[int(len(ttfts) * 0.99)],
    )
```

The p99 (99th percentile) shows worst-case behavior. If mean TTFT is 30ms but p99 is 500ms, one in a hundred users waits half a second. That matters for user experience.

Expected Results

For a 7B model on H100 with `batch_size=32`:

Throughput: 4000-6000 tok/s

TTFT p50: 30-60 ms

TTFT p99: 100-250 ms

ITL p50: 5-10 ms

ITL p99: 15-25 ms

These numbers assume the server is properly warmed up and the benchmark runs long enough to reach steady state. Short benchmarks (under 30 seconds) often show artificially good results because the system has not hit memory limits yet.

Interpreting Problems

High TTFT p99 with low p50 means some requests wait in queue while others proceed immediately. The queue backs up occasionally, likely when many requests arrive simultaneously. Solutions: increase batch size, add request shedding, or accept the variance.

High ITL p99 means token generation sometimes stalls. Common causes: Python GC pauses, scheduler lock contention, or memory allocation during generation. Profile to find the culprit.

Throughput not scaling with batch size means memory bandwidth is saturated. The GPU can only move so much data per second. You are hitting the roofline; further batching does not help.

Low GPU utilization despite high load means the bottleneck is elsewhere. Check CPU utilization (tokenization?), memory bandwidth (too much data movement?), or I/O (disk or network?).

Profiling

When benchmarks reveal problems, profiling shows where time goes. CPU profiling with cProfile:

```
import cProfile
profiler = cProfile.Profile()
profiler.enable()
run_workload()
profiler.disable()
profiler.print_stats('cumulative')
```

GPU profiling with nsight systems:

```
nsys profile -o trace python server.py &
sleep 10
curl -X POST localhost:8000/v1/completions -d '{"prompt": "Hello"}'
nsys stats trace.nsys-rep
```

The nsys trace shows kernel execution times, memory transfers, and gaps between operations. Look for unexplained gaps: those are where the GPU sits idle waiting for something.

For detailed kernel analysis, nsight compute profiles individual kernel launches. This reveals whether kernels are compute-bound or memory-bound, and what optimizations might help.

See `ch10/benchmark_ch10.py` for the complete benchmarking suite including worker scaling tests, server throughput measurements, and tokenization analysis.

Appendix

This appendix covers topics that extend beyond the core material. Some are previews of advanced techniques we did not have space to cover in depth. Others are pointers to resources for continued learning.

Speculative decoding represents the next frontier of inference optimization - using a small model to draft tokens that a large model verifies in parallel. We give a preview of the technique and its implications.

The further reading section collects the papers, repositories, and blog posts that informed this book. These are your starting points for going deeper.

What's Next

You now understand the core machinery of LLM inference. This section points to areas where research and engineering continue to push boundaries.

Speculative Decoding

Decode steps are memory-bound. The GPU has spare compute capacity. Speculative decoding uses a small draft model to propose multiple tokens, then verifies them in parallel with the large model. Correct guesses save decode steps. Wrong guesses cost nothing - we just discard them.

The key insight: verification is cheap because attention over existing KV cache is memory-bound anyway. Verifying 4 speculated tokens costs about the same as generating 1.

Quantization

We briefly mentioned fp16 and int8. Production systems go further:

- INT4 and INT3 weight quantization
- FP8 for both weights and activations
- Mixed precision strategies per layer

AWQ, GPTQ, and SmoothQuant are the key techniques. They reduce memory bandwidth requirements, directly improving decode throughput.

Long Context

32K, 128K, and million-token contexts require specialized attention:

- Sliding window limits attention to recent tokens

- Hierarchical attention summarizes older context
- Sparse attention patterns reduce quadratic complexity

Memory management becomes critical when KV caches grow to tens of gigabytes.

Multimodal

Vision-language models add image encoders. The prefill phase processes image patches alongside text tokens. Decode remains the same - the model generates text conditioned on the multimodal context.

Disaggregated Serving

Prefill and decode have different characteristics - prefill is compute-bound, decode is memory-bound. Some production systems separate these phases entirely, with different hardware optimized for each. Understanding why this makes sense comes directly from the roofline analysis in Chapter 3.

Custom Silicon

NVIDIA GPUs dominate today, but custom inference chips are emerging:

- Google TPUs optimize for matrix operations
- Groq's LPU eliminates memory hierarchy complexity
- Cerebras wafer-scale chips avoid communication overhead

Each makes different tradeoffs. Understanding the fundamentals helps you evaluate them.

Where to Contribute

Open source inference is active. SGLang, vLLM, and TensorRT-LLM welcome contributions:

- Kernel optimizations for specific operations
- New scheduling algorithms
- Support for new model architectures
- Benchmarking and profiling tools

The best way to learn is to pick one subsystem and make it faster.

Speculative Decoding Preview

Speculative decoding is the most important optimization we did not cover in depth. This section gives a preview of how it works.

The core observation: decode steps are memory-bound. We read the entire model from memory to generate one token. The GPU compute units are mostly idle. What if we could generate multiple tokens per memory read?

The idea: use a small, fast draft model to propose several tokens. Then verify them all at once with the large target model. Verification uses the same forward pass structure as prefill - we process multiple tokens in parallel.

The draft phase proposes multiple tokens quickly:

```
def draft_tokens(draft_model, kv_cache_draft, num_speculate=4):
    speculated = []
    for _ in range(num_speculate):
        token = draft_model.decode_step(kv_cache_draft)
        speculated.append(token)
        kv_cache_draft.append(token)
    return speculated
```

The verification phase accepts or rejects based on probability ratios:

```
def verify_tokens(speculated, target_model, kv_cache_target, draft_probs):
    target_logits = target_model.forward(speculated, kv_cache_target)
    output = []
    for i, token in enumerate(speculated):
        ratio = softmax(target_logits[i])[token] / draft_probs[i][token]
        if random() < ratio: output.append(token)
        else: output.append(sample(target_logits[i])); break
    return output
```

The verification uses rejection sampling. If the target model assigns higher probability than the draft model, we always accept. If lower, we accept with probability proportional to the ratio. This ensures the output distribution matches the target model exactly.

Why it helps:

Without speculation:

- 1 target forward pass = 1 token
- Cost: full model memory read

With speculation (4 tokens, 3 accepted on average):

- 4 draft forward passes + 1 target forward pass = 3.5 tokens on average
- Draft model is ~10x smaller, so: $0.4 + 1.0 = 1.4$ full model equivalents
- Speedup: $3.5 / 1.4 = 2.5x$

The speedup depends on:

- Draft model quality (acceptance rate)
- Draft model size (speculation cost)
- Number of tokens speculated
- Target model memory bandwidth utilization

Production systems like those from Anthropic and OpenAI use speculative decoding. The draft model is often a distilled version of the target, optimized for high acceptance rates on the target's distribution.

Medusa and EAGLE extend this further with parallel draft heads that share the target model's body, eliminating the separate draft model entirely.

This technique represents where inference optimization is heading: using spare compute cycles to do useful work that reduces total memory traffic.

Further Reading

The papers, repositories, and blog posts that informed this book.

Foundational Papers

Attention Is All You Need (Vaswani et al., 2017) - The transformer architecture. Everything builds on this.

GPT-2 (Radford et al., 2019) - Decoder-only transformers and autoregressive generation at scale.

GQA: Training Generalized Multi-Query Transformer Models (Ainslie et al., 2023) - Grouped query attention, the standard for modern inference-optimized models.

Memory and Compute

Roofline: An Insightful Visual Performance Model (Williams et al., 2009) - The roofline model for understanding compute vs memory bounds.

FlashAttention: Fast and Memory-Efficient Exact Attention (Dao et al., 2022) - IO-aware attention that changed how we think about GPU algorithms.

FlashAttention-2: Faster Attention with Better Parallelism (Dao, 2023) - Improved work partitioning and occupancy.

Batching and Scheduling

Orca: A Serving System for Transformer-Based Generative Models (Yu et al., 2022) - Continuous batching. The paper that started modern inference systems.

vLLM: Efficient Memory Management for Large Language Model Serving (Kwon et al., 2023) - PagedAttention and KV cache memory management.

SGLang: Efficient Execution of Structured Language Model Programs (Zheng et al., 2024) - Radix-Attention and prefix caching.

Quantization

LLM.int8(): 8-bit Matrix Multiplication for Transformers (Dettmers et al., 2022) - Mixed-precision inference with outlier handling.

AWQ: Activation-aware Weight Quantization (Lin et al., 2023) - 4-bit weight quantization preserving accuracy.

GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers (Frantar et al., 2022) - One-shot weight quantization.

Speculative Decoding

Fast Inference from Transformers via Speculative Decoding (Leviathan et al., 2023) - The foundational speculative decoding paper.

Medusa: Simple LLM Inference Acceleration Framework (Cai et al., 2024) - Multiple draft heads without separate draft model.

Systems and Repositories

vLLM (github.com/vllm-project/vllm) - Production-ready inference engine with PagedAttention.

SGLang (github.com/sgl-project/sglang) - High-performance inference with RadixAttention and structured generation.

TensorRT-LLM (github.com/NVIDIA/TensorRT-LLM) - NVIDIA's optimized inference library.

FlashInfer (github.com/flashinfer-ai/flashinfer) - Customizable attention kernels for inference.

Triton (github.com/triton-lang/triton) - Python DSL for GPU kernels.

Blog Posts and Documentation

How continuous batching enables 23x throughput in LLM inference (Anyscale, 2023) - Accessible explanation of continuous batching.

A guide to LLM inference and performance (Baseten, 2024) - Practical performance tuning guide.

CUDA C++ Programming Guide (NVIDIA) - Essential reference for GPU programming.

CUTLASS Documentation (NVIDIA) - Understanding optimized matrix multiply implementations.

Courses

CUDA Programming (freeCodeCamp) - 12-hour video course covering CUDA from first principles. Available free on YouTube.

GPU Programming (Andreas Kloeckner, UIUC) - Full GPU programming course covering CUDA from basics to optimization.

CS149: Parallel Computing (Stanford) - Fundamentals of parallel systems.

These resources cover the theory. The best next step is to read the source code of vLLM, SGLang, or FlashInfer, and start contributing.